

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TÂN TẠO
KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN TỐT NGHIỆP ĐẠI HỌC
NGHIÊN CỨU VÀ XÂY DỰNG HỆ THỐNG PHÁT HIỆN
TIN GIẢ
TIẾNG VIỆT SỬ DỤNG PHOBERT VÀ MẠNG NƠ-RON
ĐA TẦNG
(Hệ thống ShieldAI)

Ngành:	Khoa Học Máy Tính
Mã số:	7480101
Khóa:	2022

Sinh viên thực hiện:	Hà Minh Chiến
Mã số sinh viên:	2202095
Giảng viên hướng dẫn:	Thầy Trần Ngọc Anh

TÂY NINH, THÁNG 06 NĂM 2026

TRANG PHÊ DUYỆT

Luận văn đã được trình và bảo vệ trước Hội đồng chấm tiểu luận tốt nghiệp.

Ngày bảo vệ:/...../2026

Kết quả:/10 điểm

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

GIẢNG VIÊN PHẢN BIỆN

(Ký và ghi rõ họ tên)

CHỦ TỊCH HỘI ĐỒNG

(Ký và ghi rõ họ tên)

LỜI CẢM ƠN

Để hoàn thành tiểu luận tốt nghiệp này, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Quý Thầy Cô Khoa Công nghệ Thông tin - Trường Đại học Tân Tạo, những người đã truyền đạt cho em lượng kiến thức quý báu và định hướng nghề nghiệp vững chắc trong suốt những năm tháng học tập tại trường.

Đặc biệt, em xin gửi lời tri ân sâu sắc đến **Thầy Trần Ngọc Anh** - người Thầy đã trực tiếp tận tình hướng dẫn, chỉ bảo và sát cánh cùng em từ những bước lên ý tưởng sơ khai cho đến khi hoàn thiện toàn bộ mã nguồn và tài liệu của tiểu luận này. Sự định hướng khoa học và những lời động viên của Thầy là động lực to lớn giúp em vượt qua những khó khăn trong quá trình nghiên cứu và xây dựng hệ thống phát hiện tin giả tiếng Việt bằng học máy.

Em cũng xin gửi lời cảm ơn đến gia đình và những người bạn đã luôn ở bên cạnh động viên, khích lệ tinh thần và tạo mọi điều kiện thuận lợi nhất để em có thể toàn tâm toàn ý hoàn thành đồ án này đúng tiến độ.

Mặc dù đã nỗ lực hết mình, nhưng do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, tiểu luận chắc chắn không thể tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những lời góp ý, chỉ bảo quý báu từ Hội đồng bảo vệ và các Thầy Cô để đề tài được hoàn thiện hơn nữa và có thể ứng dụng rộng rãi vào thực tế, góp phần xây dựng một không gian mạng Việt Nam sạch và an toàn hơn.

Cuối cùng, em xin kính chúc Quý Thầy Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp trồng người cao quý.

Tây Ninh, ngày tháng năm 20.....

Sinh viên thực hiện

TÓM TẮT TIỂU LUẬN

Bộ dữ liệu được sử dụng trong nghiên cứu gồm **10.617 bản ghi** (tệp `full_dataset.csv`), sau lọc còn **10.609 mẫu** phục vụ huấn luyện. Tỷ lệ phân bố giữa hai lớp tương đối cân bằng, góp phần giảm thiểu ảnh hưởng của hiện tượng mất cân bằng dữ liệu (Class Imbalance) trong quá trình huấn luyện mô hình. Điều này kết hợp với phương pháp kiểm định chéo 5-fold (Cross-validation) tạo điều kiện thuận lợi cho việc đánh giá hiệu năng mô hình một cách khách quan và đáng tin cậy.

Đối với bài toán xử lý ngôn ngữ tự nhiên tiếng Việt, mô hình PhoBERT được lựa chọn làm thành phần trích xuất đặc trưng ngữ nghĩa nhờ được tiền huấn luyện trên lượng lớn dữ liệu tiếng Việt. Việc sử dụng PhoBERT giúp mô hình khai thác hiệu quả các mối quan hệ ngữ cảnh giữa các từ và cụm từ trong văn bản, từ đó hỗ trợ nâng cao chất lượng biểu diễn dữ liệu đầu vào cho tác vụ phát hiện tin giả.

Bên cạnh mô hình dự đoán, nghiên cứu thiết kế một Động cơ giải thích rule-based độc lập (ExplanationEngine) dựa trên các đặc trưng kinh nghiệm (Heuristics). Thay vì tích hợp trực tiếp Heuristics vào quá trình dự đoán toán học, hệ thống chỉ chạy module này sau khi PhoBERT đã tính toán xong xác suất, nhằm sinh ra các câu văn tiếng Việt minh bạch hóa lý do bài báo bị cảnh báo (ví dụ: lạm dụng viết hoa, văn phong giật gân).

Kết quả thực nghiệm trên tập kiểm thử (30% dữ liệu) cho thấy kiến trúc PhoBERT Text-only + MLP đạt Accuracy 94,13%, F1-Score 93,71%, và ROC-AUC 98,50%. Bên cạnh đó, kết quả kiểm định chéo 5-Fold ghi nhận F1-Score ở mức $93,17\% \pm 1,33\%$. Đồng thời, việc tách biệt cơ chế Heuristics thành Động cơ giải thích độc lập cho phép hệ thống trích xuất các dấu hiệu hình thức để giải thích lý do phân loại, qua đó hạn chế tính chất "hộp đen" của mô hình học sâu.

Từ khóa: Phát hiện tin giả tiếng Việt, PhoBERT, Giải thích Heuristic, Học máy, Xử lý ngôn ngữ tự nhiên.

ABSTRACT

The dataset utilized in this study comprises **10,617 labeled records** (`full_dataset.csv`), reduced to **10,609 samples** after cleaning for training. The distribution between the two classes is relatively balanced, helping to minimize the impact of Class Imbalance during the model training phase. This establishes a favorable condition for an objective and reliable evaluation of the model's performance through 5-fold cross-validation.

For the Vietnamese Natural Language Processing task, the PhoBERT model was selected as the semantic feature extraction component due to its extensive pre-training on a large-scale Vietnamese corpus. The employment of PhoBERT enables the model to effectively exploit the contextual relationships between words and phrases, thereby enhancing the quality of input data representation for the fake news detection task.

Alongside the predictive model, the research designs an independent rule-based Explanation Engine utilizing heuristic features. Rather than embedding heuristics into the mathematical prediction process, the system executes this module after PhoBERT has calculated the probabilities. This decoupled approach generates clear, transparent textual reasons (e.g., excessive capitalization, sensationalist tone) to justify the warnings.

Experimental results demonstrate that the proposed PhoBERT Text-only + MLP architecture achieves excellent performance (93.71% F1-Score) on the research dataset. Furthermore, decoupling the heuristics into an independent Explanation Engine helps alleviate the "black-box" nature of deep learning, providing users with intuitive, text-based reasoning for the system's classification decisions.

Keywords: Fake News Detection, PhoBERT, Heuristic Explanation, Rule-based Engine, Text-only MLP, NLP.

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU	11
1.1. Đặt vấn đề	11
1.2. Mục tiêu nghiên cứu	11
1.2.1. Mục tiêu tổng quát	11
1.2.2. Mục tiêu cụ thể	11
1.3. Đối tượng và phạm vi nghiên cứu	12
1.3.1. Đối tượng nghiên cứu	12
1.3.2. Phạm vi nghiên cứu	12
1.4. Phương pháp nghiên cứu	12
1.5. Ý nghĩa khoa học và thực tiễn	12
1.6. Bố cục đồ án	12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN NGHIÊN CỨU	13
2.1. Các khái niệm nền tảng	13
2.1.1. Tin giả (Fake News) [3, 4] và Thông tin sai lệch	13
2.1.2. Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing - NLP)	13
2.1.3. Trí tuệ Nhân tạo (AI) và Học máy (Machine Learning)	13
2.1.4. Học sâu (Deep Learning) và Mạng nơ-ron (Neural Networks)	14
2.1.5. Cơ chế Giải thích dựa trên Đặc trưng (Heuristic Explanation) [5, 8]	14
2.1.6. Các chỉ số đánh giá hiệu năng mô hình (Evaluation Metrics)	14
2.2. Cơ sở lý thuyết chuyên sâu	15
2.2.1. Kiến trúc Transformer và Cơ chế Tự chú ý (Self-Attention)	15
2.2.2. Mô hình PhoBERT: Cấu trúc và phương pháp huấn luyện	16
2.2.3. Cơ sở lý thuyết của Đặc trưng kinh nghiệm (Heuristics)	16
2.2.4. Mạng Nơ-ron Truyền thẳng (Multi-Layer Perceptron - MLP)	16
2.2.5. Cơ sở lý thuyết về Cơ chế Giải thích dựa trên Đặc trưng (Feature-based Explanation)	17
2.3. Tổng quan các công trình nghiên cứu liên quan	17
2.3.1. Các nghiên cứu trong nước	17
2.3.2. Các nghiên cứu ngoài nước	18
2.3.3. So sánh tính ưu việt của đồ án (ShieldAI) so với các hệ thống hiện tại	18
2.4. Công nghệ và Công cụ sử dụng	20
2.4.1. Thư viện Trí tuệ Nhân tạo và Xử lý Ngôn ngữ	20
2.5. Biện luận lựa chọn kiến trúc AI	20
2.5.1. Lựa chọn Mô hình Ngôn ngữ Lỗi (PhoBERT)	20
2.5.2. Lựa chọn Kiến trúc PhoBERT Text-only + MLP	21
2.5.3. Lựa chọn cơ chế giải thích dựa trên đặc trưng	21
2.6. Kết luận chương	21
3.1. Phương pháp nghiên cứu	23
3.1.1. Phân tích Dữ liệu nghiên cứu (Dataset Analysis)	23
3.1.2. Tiền xử lý và Làm sạch Dữ liệu (Data Cleaning)	23
3.1.3. Phân chia Dữ liệu Huấn luyện (Data Split)	24
3.1.4. Phương pháp Trích xuất biểu diễn ngữ nghĩa bằng PhoBERT	24

3.1.5. Phương pháp Mô hình hóa mạng đa tầng (PhoBERT + MLP)	24
3.1.6. Phương pháp Đánh giá và Giải thích (Explainability Integration)	25
3.2. Công nghệ triển khai hệ thống phần mềm	25
3.3. Phân tích yêu cầu	26
3.3.1. Yêu cầu chức năng	26
3.3.2. Yêu cầu phi chức năng	27
3.4. Thiết kế hệ thống	28
3.4.1. Kiến trúc hệ thống	28
3.4.2. Thiết kế cơ sở dữ liệu và Mô hình Thực thể - Mối quan hệ (ERD)	31
3.4.3. Thiết kế giao diện (UI/UX Design) và Trải nghiệm Người dùng	35
3.4.4. Thiết kế thuật toán (Thuật toán PhoBERT + MLP - PhoBERT + MLP)	36
3.5. Kết luận chương	42
4.1. Môi trường phát triển và Thực nghiệm	43
4.1.1. Môi trường Phần cứng (Hardware Environment)	43
4.1.2. Môi trường Hệ điều hành và Phần mềm	43
4.2. Quá trình hiện thực	44
4.2.1. Cài đặt các module lõi của hệ thống (Core Modules Implementation)	44
4.2.2. Dữ liệu nghiên cứu và Quy trình Tiền xử lý	45
4.2.3. Tổ chức cấu trúc mã nguồn (Project Directory Structure)	47
4.3. Kết quả đạt được	48
4.3.1. Kết quả giao diện ứng dụng (User Interface Implementation)	48
4.3.2. Đánh giá kết quả bằng số liệu thực nghiệm	51
4.4. Kiểm thử hệ thống (System Testing)	53
4.4.1. Kiểm thử tự động bằng Pytest (Automated Testing)	53
4.4.1.1. Phân nhóm và chi tiết 42 test cases (Bảng 4.4)	54
4.4.2. Kiểm thử Hộp đen mức Ứng dụng (Manual Black-box Testing)	54
4.5. Kết luận chương	57
5.1. Kết luận	58
5.2. Đóng góp của tiểu luận (Tính mới & Tính Sáng tạo)	58
5.3. Hạn chế của đề tài	59
5.4. Hướng phát triển	60
TÀI LIỆU THAM KHẢO	62
PHỤ LỤC	64
Phụ lục A: Mã nguồn chương trình	64
Phụ lục B: Hướng dẫn sử dụng	65

DANH MỤC HÌNH ẢNH

Số hiệu	Tên hình ảnh	Trang
Hình 4.1	Giao diện màn hình Cổng thông tin (Home) với thiết kế tối giản, tông màu tối chuyên nghiệp	30
Hình 4.2	Giao diện Đăng nhập (Authentication) bảo mật người dùng	31
Hình 4.3	Giao diện trung tâm Phân tích — nhập văn bản thô hoặc URL để kiểm chứng	32
Hình 4.4	Báo cáo phân tích cảnh báo nguy cơ Tin Giả thông qua nguyên lý Thị giác hóa rủi ro (Màu Đỏ)	33
Hình 4.5	Báo cáo phân tích xác thực Tin Thật an toàn (Màu Xanh lá cây)	34
Hình 4.6	Trình diễn khả năng của Động cơ Giải thích: Hệ thống bóc tách rõ lý do bài báo bị đánh dấu lừa đảo	35
Hình 4.7	Màn hình Quản lý Lịch sử cá nhân với thiết kế dạng danh sách thẻ động (Dynamic Cards)	36
Hình 4.8	Minh họa Kịch bản TC_01: Phân tích bài báo chính thống (0% Tin giả)	40
Hình 4.11	Minh họa Kịch bản TC_02: Ngăn chặn lỗi khi phân tích URL sai định dạng	41
Hình 4.12	Minh họa Kịch bản TC_03: Bóc trần thủ thuật lạm dụng hình thức (100% Tin giả)	42

DANH MỤC BẢNG BIỂU

Số hiệu	Tên bảng biểu	Trang
Bảng 3.1	Yêu cầu chức năng hệ thống (FR-01 ... FR-08)	23
Bảng 3.2	Từ điển Dữ liệu (Data Dictionary) cho các Thực thể	25
Bảng 4.1	Hiệu năng của mô hình PhoBERT Text-only + MLP	38
Bảng 4.2	Danh sách Kịch bản Kiểm thử Hộp đen	39
Bảng 4.3	Bộ kiểm thử tự động Pytest — tổng quan theo file	40
Bảng 4.4	Chi tiết từng kịch bản kiểm thử tự động (42 test cases)	41

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích thuật ngữ tiếng Anh (tiếng Việt)
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
BERT	Bidirectional Encoder Representations from Transformers (Mô hình biểu diễn ngôn ngữ hai chiều từ Transformer)
DL	Deep Learning (Học sâu)
DOM	Document Object Model (Mô hình Đối tượng Tài liệu)
ERD	Entity-Relationship Diagram (Sơ đồ Thực thể - Mối quan hệ)
HTML	HyperText Markup Language (Ngôn ngữ đánh dấu siêu văn bản)
JSON	JavaScript Object Notation (Định dạng trao đổi dữ liệu)
ML	Machine Learning (Học máy)
MLP	Multilayer Perceptron (Mạng nơ-ron truyền thẳng đa tầng)
NLP	Natural Language Processing (Xử lý Ngôn ngữ Tự nhiên)
ORM	Object-Relational Mapping (Ánh xạ đối tượng - quan hệ)
UI / UX	User Interface / User Experience (Giao diện người dùng / Trải nghiệm người dùng)
URL	Uniform Resource Locator (Định vị tài nguyên thống nhất)
Giải thích Heuristic	Explainable Artificial Intelligence (Trí tuệ nhân tạo có thể giải thích / AI Minh bạch)

CHƯƠNG 1: GIỚI THIỆU

1.1. Đặt vấn đề

Sự bùng nổ của mạng xã hội đã tạo điều kiện cho "tin giả" (Fake News) lan truyền, thao túng dư luận và gây rủi ro xã hội. Dù các mô hình Học sâu như Transformer và PhoBERT đã đạt nhiều tiến bộ trong việc phân tích văn bản, chúng vẫn bộc lộ hạn chế lớn là tính chất "hộp đen" và thiếu khả năng diễn giải kết quả cho người dùng cuối. Do đó, đề tài **"Nghiên cứu và xây dựng hệ thống phát hiện tin giả tiếng Việt sử dụng PhoBERT và mạng nơ-ron đa tầng (Hệ thống ShieldAI)"** được thực hiện nhằm giải quyết vấn đề này.

Đề tài tập trung nghiên cứu và xây dựng hệ thống phân loại tin giả dựa trên mô hình ngôn ngữ tiên huấn luyện PhoBERT kết hợp với mạng nơ-ron truyền thẳng (MLP). Hệ thống này tập trung khai thác các đặc trưng ngữ nghĩa từ văn bản để đưa ra phán quyết chuẩn xác. Đặc biệt, nhằm tăng khả năng giải thích quyết định của hệ thống và cải thiện khả năng giải thích của mô hình, nghiên cứu đề xuất tích hợp một Động cơ giải thích (Explanation Engine) độc lập dựa trên luật và các đặc trưng kinh nghiệm (Heuristics) — nhằm bóc tách các dấu hiệu thao túng cảm xúc, văn phong giật gân, sự lạm dụng dấu câu — từ đó góp phần nâng cao khả năng giải thích cho quyết định phân loại của mô hình.

Để đánh giá tính khả thi của phương pháp đề xuất, mô hình được triển khai và kiểm thử trên bộ dữ liệu tin tức tiếng Việt đã được gán nhãn. Các chỉ số đánh giá cốt lõi như Accuracy, Precision, Recall và F1-Score được sử dụng để phân tích hiệu năng của mạng phân loại. Kết quả thực nghiệm, tính hiệu quả của cơ chế giải thích cùng các phân tích chi tiết sẽ được trình bày trong chương đánh giá của tiểu luận.

1.2. Mục tiêu nghiên cứu

1.2.1. Mục tiêu tổng quát

Phát triển nền tảng Web kiểm duyệt tin giả tiếng Việt tự động, ứng dụng kiến trúc AI tinh gọn (PhoBERT Text-only + MLP) và cơ chế giải thích dựa trên đặc trưng để minh bạch hóa quyết định, đồng thời quản lý lịch sử tra cứu của người dùng an toàn.

1.2.2. Mục tiêu cụ thể

- **Hệ thống hóa lý thuyết:** Tổng hợp tri thức về NLP, mô hình PhoBERT, phân loại MLP (scikit-learn) và giải thích rule-based (XAI).
- **Tối ưu mô hình PhoBERT + MLP:** Sử dụng vector [CLS] 768 chiều duy nhất từ PhoBERT kết hợp mạng MLP (128, 64) để tăng độ chính xác phân loại.
- **Xây dựng Động cơ giải thích (Explanation Engine):** Thiết kế module rule-based độc lập chạy sau khi mô hình dự đoán xong, nhằm sinh ra câu văn tiếng Việt minh bạch hóa lý do bài báo bị cảnh báo.
- **Phát triển nền tảng phần mềm hoàn chỉnh:** Thiết kế và triển khai kiến trúc Client-Server chuyên nghiệp để biến mô hình AI nghiên cứu thành sản phẩm thực tiễn (ShieldAI), mang lại trải nghiệm tương tác liền mạch cho người dùng đầu cuối.
- **Đánh giá mô hình:** So sánh hiệu năng mô hình đề xuất với các thuật toán truyền thống (Decision Tree, Naive Bayes).

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Bao gồm: (1) Mô hình ngôn ngữ lớn PhoBERT; (2) Bộ phân loại MLP (scikit-learn); (3) Pipeline tiền xử lý preprocess_text; (4) Giải thích rule-based (ExplanationEngine); và (5) Công nghệ Web (FastAPI, Next.js, SQLite).

1.3.2. Phạm vi nghiên cứu

- **Mục tiêu phân loại:** Thay vì tiếp cận theo hướng nhị phân cứng nhắc, hệ thống ShieldAI cung cấp trải nghiệm người dùng (UX) tinh tế hơn với 3 mức độ cảnh báo: Tin thật (Xác suất < 35%), Đáng ngờ (35% - 74%), và Tin giả (Xác suất $\geq$ 75%).
- **Thuật toán:** Tập trung vào kiến trúc PhoBERT Text-only + MLP, lấy vector [CLS] 768 chiều từ PhoBERT đưa qua mạng nơ-ron truyền thẳng đa tầng.
- **Cấu trúc Triển khai ứng dụng (Deployment):** Đồ án được hệ thống hóa hoàn chỉnh theo kiến trúc Client-Server chuyên nghiệp. Giao diện người dùng (Frontend) được phát triển bằng Next.js 14 mang lại trải nghiệm tương tác mượt mà; Máy chủ (Backend) sử dụng FastAPI bất đồng bộ cho hiệu năng cao; Dữ liệu lịch sử quét tin giả được lưu trữ an toàn bằng SQLite. Đặc biệt, hệ thống tích hợp Trình thu thập dữ liệu (Web Crawler) bằng BeautifulSoup giúp tự động cào nội dung bài báo trực tiếp từ URL, tối ưu hóa đáng kể trải nghiệm người dùng (UX) thay vì ép người dùng dán văn bản thuần túy.

1.4. Phương pháp nghiên cứu

Để giải quyết toàn diện bài toán phát hiện tin giả và xây dựng một ứng dụng hoàn chỉnh, đồ án áp dụng kết hợp các phương pháp nghiên cứu sau:

- **Phương pháp nghiên cứu tài liệu (Literature Review):** Khảo sát, tổng hợp và phân tích các bài báo khoa học, tài liệu chuyên ngành về Xử lý ngôn ngữ tự nhiên (NLP), kiến trúc Transformer (đặc biệt là họ mô hình PhoBERT), và các thuật toán phân loại Học máy. Phương pháp này giúp xây dựng cơ sở lý thuyết cho đồ án.
- **Phương pháp thu thập và tiền xử lý dữ liệu (Data Preprocessing):** Xây dựng các kịch bản thu thập dữ liệu (Web Crawler) tự động hóa và áp dụng kỹ thuật làm sạch văn bản, chuẩn hóa ngôn ngữ tiếng Việt (sử dụng thư viện PyVi) để tạo ra tập ngữ liệu đầu vào chất lượng, khử nhiễu trước khi đưa vào huấn luyện mô hình.
- **Phương pháp thực nghiệm và mô hình hóa (Experimental & Modeling):** Thiết kế, xây dựng và tinh chỉnh mạng nơ-ron đa tầng (MLP) kết hợp với bộ trích xuất đặc trưng PhoBERT. Đồng thời, nghiên cứu và lập trình Cơ chế giải thích dựa trên đặc trưng hình thức (Heuristic Explanation Engine) để minh bạch hóa quá trình ra quyết định của trí tuệ nhân tạo.
- **Phương pháp thống kê và đánh giá (Statistical Evaluation):** Sử dụng phương pháp kiểm chứng chéo phân tầng (Stratified K-Fold) và các độ đo tiêu chuẩn trong Học máy (Precision, Recall, F1-Score, Ma trận nhầm lẫn - Confusion Matrix) để đánh giá định lượng hiệu năng mô hình một cách khách quan.
- **Phương pháp kỹ nghệ phần mềm (Software Engineering):** Áp dụng quy trình phát triển hiện đại để thiết kế kiến trúc phân tán 3 tầng (Client-Server), lập trình giao diện người dùng (Next.js) và tích hợp API máy chủ (FastAPI), qua đó hiện thực hóa mô hình AI thành một phần mềm thực tiễn.

1.5. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học: Đồ án cung cấp một hướng tiếp cận hiệu quả trong việc kết hợp khả năng biểu diễn ngữ nghĩa của mô hình ngôn ngữ tiếng Việt (PhoBERT) với bộ phân loại học sâu truyền thống (MLP) trên tập ngữ

liệu tiếng Việt đặc thù. Đặc biệt, việc tích hợp phương pháp Giải thích Heuristic góp phần cải thiện khả năng giải thích của mô hình, đóng góp thêm tài liệu tham khảo giá trị cho lĩnh vực Trí tuệ Nhân tạo Minh bạch (Explainable AI - XAI).

Về mặt thực tiễn: Đề tài đã xây dựng thành công nền tảng phần mềm ShieldAI hoàn thiện. Đây là một công cụ hỗ trợ kiểm chứng thông tin, cung cấp cho cộng đồng mạng Việt Nam khả năng nhận diện tin giả nhanh chóng kèm theo những lời giải thích dễ hiểu. Bên cạnh đó, với kiến trúc API linh hoạt, hệ thống có tiềm năng rất lớn để mở rộng và tích hợp vào các mạng xã hội hoặc các công cụ kiểm duyệt nội dung tự động của các cơ quan báo chí trong tương lai.

1.6. Bố cục đồ án

Ngoài phần Lời mở đầu và Kết luận, nội dung chính của đồ án được tổ chức thành 5 chương cốt lõi như sau:

- **Chương 1 - Mở đầu:** Trình bày bối cảnh, lý do chọn đề tài, mục tiêu, phạm vi nghiên cứu, các phương pháp tiếp cận và ý nghĩa tổng thể của dự án.
- **Chương 2 - Cơ sở lý thuyết và tổng quan nghiên cứu:** Cung cấp các khái niệm nền tảng về bài toán tin giả, xử lý ngôn ngữ tự nhiên, kiến trúc mô hình PhoBERT, thuật toán MLP và các độ đo đánh giá mô hình.
- **Chương 3 - Phân tích thiết kế hệ thống:** Đi sâu vào kiến trúc phần mềm, mô hình dữ liệu (ERD), và chi tiết luồng dữ liệu (Data Pipeline) từ giai đoạn thu thập văn bản đến động cơ Giải thích Heuristic.
- **Chương 4 - Kết quả thực nghiệm:** Công bố các số liệu huấn luyện mô hình, kết quả kiểm thử hệ thống tự động (Unit Test) và minh họa giao diện sử dụng thực tế của phần mềm ShieldAI.
- **Chương 5 - Tổng kết và hướng phát triển:** Đúc kết lại những thành tựu đã đạt được, chỉ ra các hạn chế còn tồn đọng và đề xuất các hướng nâng cấp hệ thống trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN NGHIÊN CỨU

2.1. Các khái niệm nền tảng

Để xây dựng một kiến trúc hệ thống phân tích tin giả phức tạp, việc nắm bắt các khái niệm cốt lõi mang tính chất nền tảng là điều kiện tiên quyết. Mục này sẽ trình bày và định nghĩa một cách khoa học các thuật ngữ chuyên ngành được sử dụng xuyên suốt trong đồ án.

2.1.1. Tin giả (Fake News) [3, 4] và Thông tin sai lệch

Trong kỷ nguyên số, **Tin giả (Fake News)** [3, 4] được định nghĩa là những thông tin sai sự thật, bị bóp méo, hoặc bịa đặt hoàn toàn, được trình bày dưới hình thức tin báo chí nhằm mục đích lừa dối người đọc. Điểm khác biệt mấu chốt của tin giả so với các lỗi sai sót báo chí thông thường nằm ở **động cơ (intent)**. Tin giả được tạo ra một cách có chủ đích nhằm thu lợi ích tài chính (thông qua việc thu hút lượt nhấp chuột - clickbait để lấy doanh thu quảng cáo) hoặc phục vụ các mục đích chính trị, thao túng tâm lý và định hướng dư luận xã hội.

Hội đồng Châu Âu (Council of Europe) phân loại thông tin sai lệch thành ba nhóm chính: *Mis-information* (Thông tin sai nhưng không có ác ý), *Dis-information* (Thông tin sai và có ác ý, bịa đặt), và *Mal-information* (Thông tin có thật nhưng được dùng để bôi nhọ, hãm hại người khác). Đồ án này tập trung chủ yếu vào việc tự động nhận diện và phát hiện nhóm *Dis-information*, đặc biệt là các bài viết sử dụng ngôn từ kích động và cấu trúc câu lừa đảo trên mạng xã hội tiếng Việt.

2.1.2. Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing - NLP)

Xử lý Ngôn ngữ Tự nhiên (NLP) là một phân ngành giao thoa giữa Khoa học Máy tính, Trí tuệ Nhân tạo và Ngôn ngữ học. Mục tiêu của NLP là thu hẹp khoảng cách giao tiếp giữa con người và máy tính bằng cách cung cấp cho máy tính khả năng đọc, hiểu, phân tích và trích xuất ý nghĩa từ ngôn ngữ tự nhiên của con người.

Trong khuôn khổ bài toán phát hiện tin giả, NLP đóng vai trò xương sống. Văn bản thô ban đầu là một dạng dữ liệu phi cấu trúc (unstructured data) mà máy tính không thể tính toán trực tiếp. Thông qua các kỹ thuật NLP như làm sạch văn bản (text cleaning), tách từ (tokenization), và biểu diễn từ ngữ dưới dạng không gian vector toán học (word embeddings), NLP biến đổi ngôn ngữ loài người thành các ma trận số liệu có cấu trúc để đưa vào huấn luyện mạng nơ-ron.

2.1.3. Trí tuệ Nhân tạo (AI) và Học máy (Machine Learning)

Trí tuệ Nhân tạo (AI) là thuật ngữ rộng chỉ khả năng của máy móc mô phỏng các chức năng nhận thức của con người như học tập và giải quyết vấn đề. Nằm bên trong lõi của AI hiện đại là **Học máy (Machine Learning - ML)**. Thay vì được lập trình bằng các quy tắc logic "If-Else" cứng nhắc do lập trình viên định sẵn, Học máy cho phép các thuật toán tự động nhận diện các mẫu (patterns) ẩn sâu trong một khối lượng dữ liệu khổng lồ, từ đó tự đúc kết quy luật để đưa ra dự đoán cho các dữ liệu mới trong tương lai.

Bài toán phát hiện tin giả trong đồ án thuộc phân lớp bài toán **Học có giám sát (Supervised Learning)**. Về cơ bản, hệ thống được cung cấp hàng chục ngàn bài báo đã được con người dán nhãn trước để huấn luyện thuật toán. Tuy nhiên, thay vì áp dụng một ranh giới quyết định (decision boundary) nhị phân cứng nhắc (Thật/Giả) ở bước dự đoán cuối cùng, hệ thống chia xác suất thành 3 mức độ cảnh báo (Verdict) linh hoạt để tăng tính thân thiện và chính xác trong trải nghiệm người dùng (UX).

2.1.4. Học sâu (Deep Learning) và Mạng nơ-ron (Neural Networks)

Học sâu (Deep Learning) là một tập con tiên tiến nhất của Học máy, lấy cảm hứng từ cấu trúc phân tầng và hoạt động của bộ não sinh học. Các hệ thống học sâu sử dụng **Mạng nơ-ron nhân tạo (Artificial Neural Networks)** với hàng chục đến hàng trăm lớp ẩn (hidden layers) được xếp chồng lên nhau. Ưu điểm của Học sâu nằm ở chỗ nó có thể tự động trích xuất các biểu diễn đặc trưng (feature representation) ở mức độ trừu

tượng rất cao từ dữ liệu thô mà không cần hoặc cần rất ít sự can thiệp kỹ thuật thủ công từ con người (feature engineering).

2.1.5. Cơ chế Giải thích dựa trên Đặc trưng (Heuristic Explanation) [5, 8]

Mặc dù Mạng nơ-ron Học sâu mang lại độ chính xác cao, chúng lại mắc phải điểm yếu chí mạng là tính chất "**Hộp đen**" (**Black-box**). Nghĩa là, kỹ sư có thể nạp dữ liệu vào và nhận kết quả đầu ra rất chuẩn xác, nhưng không ai có thể truy xuất ngược quá trình tính toán hàng triệu tham số bên trong để hiểu tại sao mô hình lại đi đến quyết định đó.

Để giải quyết vấn đề niềm tin và tính đạo đức này, khái niệm **Cơ chế Giải thích dựa trên Đặc trưng (Heuristic Explanation) [5, 8]** ra đời. Giải thích Heuristic tập hợp các thuật toán nhằm minh bạch hóa kết quả đầu ra của Học sâu, hỗ trợ người dùng diễn giải kết quả đầu ra. Trong hệ thống phát hiện tin giả, Giải thích Heuristic là cơ sở lý thuyết để xây dựng nên Động cơ giải thích, giúp máy tính trả lời được câu hỏi mang tính phản biện: "*Những đặc trưng hoặc từ vựng cụ thể nào đã khiến văn bản này bị phân loại là giả mạo?*".

2.1.6. Các chỉ số đánh giá hiệu năng mô hình (Evaluation Metrics)

Để đo lường một cách khách quan năng lực phân loại của hệ thống, đồ án sử dụng các thang đo chuẩn quốc tế dựa trên **Ma trận nhầm lẫn (Confusion Matrix)**. Ma trận này phân tách kết quả thành 4 nhóm: *True Positive - TP* (Hệ thống bắt đúng tin giả), *True Negative - TN* (Hệ thống nhận diện đúng tin thật), *False Positive - FP* (Hệ thống nhận diện nhầm tin thật thành tin giả), và *False Negative - FN* (Hệ thống bỏ sót tin giả). Từ ma trận này, đồ án sử dụng 4 chỉ số cốt lõi:

- **Độ chính xác tổng thể (Accuracy):** Là tỷ lệ phần trăm số bài báo (cả thật và giả) được hệ thống phân loại đúng trên tổng số các bài báo trong tập kiểm thử. Tuy nhiên, trong môi trường thực tế (nơi lượng tin thật thường áp đảo tin giả), Accuracy không phải là thước đo duy nhất đáng tin cậy.
- **Độ chuẩn xác (Precision):** Trả lời cho câu hỏi: "*Trong số những bài báo mà hệ thống dán nhãn là tin giả, có bao nhiêu phần trăm thực sự là tin giả?*". Chỉ số này đặc biệt quan trọng để hệ thống giảm thiểu tối đa *False Positive* (ngăn chặn việc vu khống hoặc kiểm duyệt nhầm các bài báo chân chính).
- **Độ bao phủ (Recall / Sensitivity):** Trả lời cho câu hỏi: "*Trong tổng số toàn bộ tin giả đang tồn tại thực tế, hệ thống đã bắt thành công bao nhiêu phần trăm?*". Recall cao đồng nghĩa với việc hệ thống có khả năng càn quét tốt, giảm thiểu *False Negative* (không để lọt lưới tin độc hại).
- **Điểm F1 (F1-Score):** Là trung bình điều hòa (Harmonic Mean) giữa Precision và Recall. Đây là chỉ số tối thượng (Ultimate Metric) được đồ án sử dụng làm tiêu chuẩn chính để quyết định mô hình nào hoạt động tốt nhất. F1-Score đảm bảo sự cân bằng hiệu quả, buộc hệ thống vừa không được phép "bắt nhầm" (Precision cao) vừa không được "bỏ sót" (Recall cao).

2.2. Cơ sở lý thuyết chuyên sâu

Nếu mục 2.1 dừng lại ở việc định nghĩa các khái niệm nền tảng, thì mục này sẽ đi sâu vào việc mổ xẻ cấu trúc toán học và logic vận hành bên trong của các thành phần cốt lõi cấu thành nên hệ thống AI (PhoBERT Text-only + MLP).

2.2.1. Kiến trúc Transformer và Cơ chế Tự chú ý (Self-Attention)

Trước khi kiến trúc Transformer [6, 7] ra đời (được giới thiệu bởi Vaswani et al., 2017), các bài toán NLP chủ yếu dựa vào mạng nơ-ron hồi quy (RNN) hoặc bộ nhớ ngắn hạn dài (LSTM). Hạn chế lớn nhất của RNN/LSTM là tính toán tuần tự, dẫn đến hiện tượng "quên" ngữ cảnh đối với các câu văn dài và không thể tính toán song song. Transformer ra đời đã khắc phục hạn chế này bằng cách thay thế cấu trúc hồi quy, thay vào đó sử dụng độc quyền cơ chế **Tự chú ý (Self-Attention)**.

Cơ sở toán học của Self-Attention xoay quanh ba ma trận: *Query (Q)*, *Key (K)*, và *Value (V)*. Với mỗi từ trong câu, mô hình sẽ tính toán điểm số chú ý (Attention Score) của từ đó so với tất cả các từ khác trong cùng một ngữ cảnh thông qua việc nhân ma trận (Dot-product) và áp dụng hàm Softmax. Điểm số này giúp mô hình hiểu được tầm quan trọng và sự phụ thuộc về mặt ngữ pháp/ngữ nghĩa lẫn nhau giữa các từ, bất kể khoảng cách vật lý của chúng trong câu là bao xa. Cấu trúc này cho phép máy tính xử lý song song toàn bộ văn bản, tạo ra những ma trận biểu diễn (embeddings) chất lượng cao.

2.2.2. Mô hình PhoBERT: Cấu trúc và phương pháp tiền huấn luyện

PhoBERT là một mô hình ngôn ngữ tiền huấn luyện tiên phong được huấn luyện dành riêng cho tiếng Việt. Về mặt kiến trúc cốt lõi, PhoBERT không kế thừa nguyên bản BERT của Google, mà dựa trên biến thể được tối ưu hóa mạnh mẽ hơn là **RoBERTa (Robustly Optimized BERT Pretraining Approach)**.

Khác biệt chuyên sâu của PhoBERT nằm ở chiến lược huấn luyện. Trong khi BERT sử dụng kỹ thuật *Masked Language Model (MLM)* tĩnh (các từ bị che giấu cố định trong suốt quá trình huấn luyện), PhoBERT áp dụng **MLM động (Dynamic Masking)**. Nghĩa là, ở mỗi chu kỳ huấn luyện (epoch), các từ bị che khuất sẽ thay đổi liên tục một cách ngẫu nhiên. Điều này buộc mạng nơ-ron phải học cách dự đoán từ còn thiếu dựa trên các cấp độ từ vựng ghép của tiếng Việt. Được tiền huấn luyện trên khối dữ liệu 20GB, PhoBERT sở hữu khả năng nắm bắt cấu trúc ngữ pháp, từ ghép và ngữ cảnh phức tạp của tiếng Việt.

2.2.3. Cơ sở lý thuyết của Đặc trưng kinh nghiệm (Heuristics)

Trong bài toán phân loại thông tin độc hại, nếu PhoBERT đảm nhiệm phần "Ngữ nghĩa học" (Semantics) để phân loại, thì các đặc trưng Heuristics đảm nhiệm phần "Hành vi học" (Behavioral Analysis) để giải thích kết quả. Cơ sở lý thuyết của việc sử dụng Heuristics độc lập này bắt nguồn từ việc tin giả thường có cấu trúc *thiết kế có chủ ý* để thao túng cảm xúc người đọc, và việc chỉ ra các thủ thuật này giúp người dùng dễ hiểu lý do cảnh báo hơn.

- **Phân tích Cảm xúc (Sentiment Analysis):** Các bài báo mạo danh thường lạm dụng các từ ngữ mang sắc thái cảm xúc cực đoan (kích động thù hằn, tạo sự hoang mang, sợ hãi) để ép buộc người đọc phải chia sẻ (share) ngay lập tức. Việc định lượng điểm số tiêu cực/tích cực (Polarity Score) đóng vai trò như một mỏ neo nhận diện.
- **Kỹ thuật Clickbait:** Tin giả thường sử dụng thủ thuật viết hoa toàn bộ tựa đề (CAPS LOCK) hoặc lạm dụng dấu câu (ví dụ: !!!, ???) nhằm thu hút sự chú ý của người đọc. Hệ thống sử dụng Biểu thức chính quy (Regular Expression) để trích xuất và đo lường mật độ của các dấu hiệu bất thường này.

2.2.4. Mạng Nơ-ron Truyền thẳng (Multi-Layer Perceptron - MLP)

Hệ thống sử dụng kiến trúc **Mạng nơ-ron truyền thẳng (Multi-Layer Perceptron - MLP)** để làm bộ phân loại lỗi. Lớp MLP này nhận đầu vào là vector nhúng ngữ nghĩa duy nhất gồm 768 chiều được trích xuất từ token [CLS] của mô hình PhoBERT. Vector này được chuẩn hóa (StandardScaler) trước khi đưa vào các lớp ẩn.

Cấu trúc MLP sử dụng `sklearn.neural_network.MLPClassifier` với lớp ẩn (**128, 64**), hàm kích hoạt ReLU, solver Adam, `alpha=0.1` (chuẩn hóa L2) và `early_stopping=True` để chống overfitting. `predict_proba` trả xác suất tin giả; module `verdict.py` quy đổi thành 3 mức cảnh báo: Tin thật (<35%), Đáng ngờ (35-74%), Tin giả (≥75%).

2.2.5. Cơ sở lý thuyết về Cơ chế Giải thích dựa trên Đặc trưng (Feature-based Explanation)

Bên cạnh việc đưa ra kết quả phân loại, hệ thống còn được thiết kế một Động cơ giải thích rule-based độc lập (ExplanationEngine) nhằm cung cấp các thông tin hỗ trợ người dùng hiểu rõ hơn lý do của dự đoán. Thay vì sử dụng các kỹ thuật Giải thích Trí tuệ nhân tạo (Explainable AI - XAI) phức tạp như SHAP hay LIME để mổ xẻ nội bộ mạng nơ-ron, nghiên cứu áp dụng phương pháp giải thích bằng luật dựa trên các đặc trưng kinh nghiệm (Heuristics) chạy độc lập sau bước phân loại.

Cơ chế này hoạt động bằng cách quét qua bài báo để phân tích các đặc trưng hình thức như tỷ lệ chữ in hoa, số lượng dấu chấm than, độ dài tiêu đề. Khi một đặc trưng vượt quá ngưỡng đã được lập trình sẵn, hệ thống sẽ tự động sinh ra các câu văn tiếng Việt (ví dụ: "Tiêu đề bài báo lạm dụng viết hoa") để trình bày trực tiếp cho người dùng.

Phương pháp này hoạt động độc lập với mạng nơ-ron lõi, không làm ảnh hưởng đến thời gian dự đoán của PhoBERT, mà vẫn giúp nâng cao khả năng giải thích của hệ thống. Nhờ đó, người dùng nhận được các thông tin hỗ trợ quá trình ra quyết định bằng ngôn ngữ tự nhiên thay vì những biểu đồ XAI phức tạp.

2.3. Tổng quan các công trình nghiên cứu liên quan

2.3.1. Các nghiên cứu trong nước

- *ReINTEL Challenge 2020: Vietnamese Fake News Detection using Ensemble Model with PhoBERT embeddings* (Nhiều tác giả, Hội thảo VLSP 2020): Nghiên cứu này đã ứng dụng thành công mô hình ngôn ngữ PhoBERT để trích xuất đặc trưng văn bản, kết hợp với các thuật toán Ensemble (StackNet, LightGBM) để đạt F1-Score lên tới 94% trên tập dữ liệu ReINTEL, khẳng định khả năng của PhoBERT trong bài toán tiếng Việt.
- *Phát hiện tự động tin giả: Thành tựu và thách thức* (Tạp chí Khoa học và Công nghệ - Đại học Đà Nẵng, 2021): Công trình đã hệ thống hóa và so sánh hiệu năng của hàng loạt phương pháp học máy (BoW, TF-IDF, SVM) và học sâu (CNN, LSTM) trên ngữ liệu tiếng Việt, chỉ ra giới hạn về bộ nhớ ngữ cảnh của các mạng nơ-ron thế hệ cũ khi xử lý văn bản dài.
- *Fake News Detection using PhoBERT and BiLSTM with Handcrafted Features* (Nhóm nghiên cứu thuộc các trường Đại học tại Việt Nam, 2022): Nghiên cứu đã đề xuất PhoBERT + MLP ghép thành công giữa mô hình PhoBERT [1] và mạng hồi quy BiLSTM, đồng thời chứng minh việc bổ sung các đặc trưng đếm thủ công (độ dài văn bản, thời gian đăng) giúp cải thiện đáng kể khả năng nhận diện tin giả so với việc chỉ dùng văn bản thô.
- *Nhận diện thông tin sai lệch đa phương thức trên mạng xã hội tiếng Việt* (Các nghiên cứu tham gia VLSP ReINTEL): Các nghiên cứu tiên phong đã xây dựng thành công mô hình trích xuất đặc trưng song song từ cả văn bản (PhoBERT) và hình ảnh đính kèm (CNN/Swin Transformer), mở ra hướng đi mới trong việc phát hiện tin giả dạng ảnh chế (meme) trên Facebook.

2.3.2. Các nghiên cứu ngoài nước

- *Fake News Detection on Social Media: A Data Mining Perspective* (Kai Shu et al., 2017): Nghiên cứu đã định nghĩa và phân tách thành công bộ đặc trưng phát hiện tin giả thành hai nhóm lõi: Đặc trưng nội dung (văn phong giật gân, độ dài) và Đặc trưng mạng xã hội (tương tác người dùng). Đây là công trình đặt nền móng cho việc sử dụng Heuristics trong phân tích tin giả.
- *Emotion shapes the diffusion of moralized content in social networks* (William J. Brady et al., Tạp chí PNAS, 2017): Nhóm tác giả đã chứng minh bằng thực nghiệm thống kê rằng các bài đăng chứa từ vựng mang cảm xúc phẫn nộ và sợ hãi có tốc độ lan truyền nhanh hơn tin thật gấp nhiều lần. Kết quả này xác thực tầm quan trọng của tính năng Phân tích cảm xúc (Sentiment Analysis).
- *dFEND: Explainable Fake News Detection* (Kai Shu et al., Hội nghị KDD 2019): Nghiên cứu đã xây dựng thành công một mô hình không chỉ phân loại tin giả mà còn sử dụng cơ chế Attention để tự động bôi đậm (highlight) các câu văn lừa đảo. Đây là một trong những công trình đầu tiên hiện thực hóa khái niệm Heuristic Explanation trong lĩnh vực tin tức.
- *Fake News Detection Using BERT Model with Handcrafted Features* (IEEE, 2021): Công trình đã chứng minh tính hiệu quả của kiến trúc Hybrid khi trích xuất vector ngữ nghĩa từ mô hình ngôn ngữ tiên huấn luyện (BERT) kết nối trực tiếp với ma trận đặc trưng thủ công (số lượng ký tự in hoa, dấu chấm than), giúp cải thiện rõ rệt độ chính xác của phân loại nhị phân.

2.3.3. So sánh tính ưu việt của đồ án (ShieldAI) so với các hệ thống hiện tại

Dựa trên bức tranh tổng quan của các nghiên cứu đi trước, đồ án **ShieldAI** không chỉ kế thừa mà còn giải quyết được những hạn chế cốt lõi của các hệ thống phát hiện tin giả truyền thống. Dưới đây là bảng so sánh làm nổi bật tính tối ưu (Optimization) và sự phát triển của đồ án:

1. Khả năng Nắm bắt Ngôn ngữ

- **Hệ thống đoạt giải VLSP ReINTEL Challenge 2020:** Sử dụng PhoBERT kết hợp với Ensemble Learning chạy trên tập dữ liệu tổng hợp. Tuy đạt điểm số thi đấu cao nhưng cấu trúc này dễ bị thiếu sót từ vựng khi áp dụng vào các bài báo y khoa có thuật ngữ chuyên sâu.
- **ShieldAI (Tính Khái quát hóa Cao):** Cũng kế thừa khả năng của **PhoBERT** và được huấn luyện trên bộ ngữ liệu tin y tế/sức khỏe tiếng Việt (**10.609 mẫu** sau lọc). Mô hình được trang bị khả năng hiểu rõ văn cảnh tiếng Việt, ngữ pháp tiếng lóng và các từ khóa thao túng tâm lý, hỗ trợ phát hiện các thủ đoạn lừa đảo tinh vi trên không gian mạng.

2. Kiến trúc Mô hình

- **Hệ thống đoạt giải VLSP ReINTEL Challenge 2020:** Xây dựng kiến trúc Ensemble kết hợp hàng loạt mạng Nơ-ron lớn với nhau. Mô hình này phức tạp và đòi hỏi nhiều tài nguyên, khó có khả năng ứng dụng thực tế do yêu cầu phần cứng cao.
- **ShieldAI (Tối ưu Cấu trúc Text-only):** Thiết kế cấu trúc PhoBERT + MLP tinh gọn tối đa. Thay vì nối vector phức tạp, hệ thống chỉ trích xuất vector ngữ nghĩa 768 chiều từ PhoBERT, chuẩn hóa bằng StandardScaler và phân loại qua MLP (128, 64). Cơ chế Heuristics được tách rời thành Động cơ giải thích độc lập. Điều này giúp hệ thống đạt F1-Score 93,71% trong khi vẫn tối ưu hóa thời gian suy luận trên môi trường Web.

3. Tính Minh bạch (Explainability)

- **Hệ thống đoạt giải VLSP ReINTEL Challenge 2020:** Hoạt động như một "Hộp đen" (Black-box) 100%. Khi nhận đầu vào, mô hình chỉ xuất ra một tệp kết quả chứa nhãn "0" (Thật) hoặc "1" (Giả) mà không thể đưa ra bất kỳ lý do giải thích cho kết quả đó.
- **ShieldAI (Tối ưu Niềm tin):** Tích hợp hệ thống **Động cơ Giải thích Heuristic** (rule-based). Phần mềm quét các tín hiệu hình thức trên văn bản gốc và in ra màn hình lý do cảnh báo (ví dụ: "Tiêu đề viết hoa quá nhiều" hay "Văn phong giật gân, thiếu tính trung lập"). Giúp người dùng thực sự tin tưởng vào phán quyết của AI.

4. Trải nghiệm & Khả năng Vận hành Thực tế

- **Hệ thống đoạt giải VLSP ReINTEL Challenge 2020:** Hoàn toàn không có giao diện người dùng (No UI) và không có khả năng tự lấy tin tức. Chỉ là một tệp mã nguồn chạy Offline nhập liệu thủ công trên máy tính cá nhân của nhà nghiên cứu.
- **ShieldAI (Tối ưu Trải nghiệm):** Đóng gói mô hình AI thành một nền tảng phần mềm Client-Server hoàn chỉnh. Tích hợp **BeautifulSoup** (Web Crawler) tự động cào nội dung tin tức trực tiếp từ URL mạng trong tích tắc thay vì copy-paste thủ công. Backend sử dụng **FastAPI** bất đồng bộ (Asynchronous) kết hợp với Frontend **Next.js 14** và lưu trữ **SQLite** nhằm mang lại hiệu năng cao và UX/UI chuyên nghiệp. Đưa AI ra khỏi phòng thí nghiệm và sẵn sàng phục vụ người dùng cuối (End-user).

2.4. Công nghệ và Công cụ sử dụng

Đồ án này không chỉ dừng lại ở việc thiết kế một mô hình AI chạy trong môi trường nghiên cứu cô lập (Jupyter Notebook), mà được xây dựng thành một nền tảng phần mềm hoàn chỉnh theo kiến trúc Client-Server. Dưới đây là các công nghệ lõi được sử dụng để hiện thực hóa toàn bộ hệ thống:

2.4.1. Thư viện Trí tuệ Nhân tạo và Xử lý Ngôn ngữ

- **PyTorch & Transformers:** PyTorch được dùng để chạy mô hình **PhoBERT-base** (embedding [CLS] 768 chiều) qua thư viện Hugging Face Transformers. Bộ phân loại **MLP** được huấn luyện và suy luận bằng `scikit-learn` (`MLPClassifier` + `StandardScaler`), lưu artifact dạng `.joblib`.
- **HuggingFace Transformers:** Là thư viện tiêu chuẩn toàn cầu trong lĩnh vực Xử lý ngôn ngữ tự nhiên. Thư viện này cho phép hệ thống nạp trực tiếp kiến trúc `vinai/phobert-base-v2` kèm theo trọng số (pre-trained weights) và Tokenizer chuẩn xác của nhóm nghiên cứu VinAI, giúp tiết kiệm hàng ngàn giờ huấn luyện lại từ đầu.
- **PyVi (Python Vietnamese Toolkit):** Đặc thù của tiếng Việt là cấu trúc từ ghép (ví dụ: "sinh viên" là một từ có nghĩa thay vì hai từ đơn). Để mô hình PhoBERT [1] không bị hiện tượng thiếu hụt từ vựng (Out-of-Vocabulary), thư viện PyVi (module `ViTokenizer`) được hệ thống sử dụng độc quyền trong pipeline tiền xử lý để tự động nối các từ ghép bằng dấu gạch dưới (ví dụ: "sinh_viên"), đảm bảo tính toàn vẹn về mặt ngữ nghĩa trước khi chuyển đổi thành vector.
- **Scikit-Learn & Pandas:** Thư viện Pandas cung cấp cấu trúc `DataFrame` giúp việc làm sạch và thao tác trên khối lượng lớn dữ liệu huấn luyện trở nên hiệu quả. Scikit-Learn được sử dụng để tính toán các ma trận đánh giá hiệu năng (Confusion Matrix, F1-Score) và hỗ trợ chia tách tập dữ liệu (train/validation/test split) một cách ngẫu nhiên và phân tầng (stratified).

2.5. Biện luận lựa chọn kiến trúc AI

Dựa trên các phân tích lý thuyết, mục này sẽ đưa ra các luận điểm chuyên sâu nhằm giải thích rõ lý do tại sao các công nghệ lỗi lại được lựa chọn cho đồ án, đồng thời so sánh trực tiếp với các phương pháp truyền thống để làm nổi bật tính tối ưu của hệ thống.

2.5.1. Lựa chọn Mô hình Ngôn ngữ Lỗi (PhoBERT)

Hệ thống quyết định sử dụng **PhoBERT** (mô hình Transformer dành riêng cho tiếng Việt) thay vì các thuật toán Machine Learning truyền thống (SVM, Naive Bayes) hay Multilingual BERT (mBERT). Tiếng Việt có cấu trúc ngữ pháp đặc thù (từ ghép, từ mượn, tiếng lóng mạng). Việc PhoBERT được huấn luyện sẵn (pre-trained) trên kho dữ liệu 20GB văn bản tiếng Việt giúp mô hình nắm bắt ngữ cảnh và hàm ý sâu xa của câu chữ. Nếu so sánh, phương pháp TF-IDF kết hợp SVM tuy có tốc độ xử lý nhanh nhưng hạn chế trong việc nắm bắt ngữ cảnh, dễ dàng bị đánh lừa bởi các thủ thuật lách luật. Trong khi đó, mBERT dù hỗ trợ đa ngôn ngữ nhưng vốn từ tiếng Việt lại quá mỏng. Do đó, PhoBERT được lựa chọn nhằm cải thiện độ chính xác (Accuracy).

2.5.2. Lựa chọn Kiến trúc PhoBERT + MLP (Text-only)

Đồ án áp dụng kiến trúc tinh gọn **PhoBERT Text-only + MLP**. Thay vì dung hợp rườm rà, mô hình chỉ sử dụng vector ngữ nghĩa của PhoBERT (768 chiều) làm đầu vào cho mạng Nơ-ron Truyền thẳng (MLP) có cấu trúc lớp ẩn (128, 64). Việc sử dụng một luồng đặc trưng giúp dễ dàng quản lý pipeline, giảm thiểu chi phí tính toán và tận dụng được khả năng của PhoBERT trong việc nắm bắt ngữ cảnh. Các đặc trưng hình thức (Heuristics) được tách hẳn ra thành một phân hệ Động cơ Giải thích (Explanation Engine) độc lập chạy sau khi phân loại. Cách tiếp cận phân tách này giúp hệ thống đạt F1-Score 93,71% trên tập kiểm thử.

2.5.3. Lựa chọn cơ chế giải thích dựa trên đặc trưng

Trong nhiều hệ thống học sâu hiện nay, kết quả phân loại thường được cung cấp dưới dạng nhãn dự đoán mà không đi kèm thông tin giải thích, gây khó khăn cho người dùng trong việc đánh giá mức độ tin cậy của kết quả. Để khắc phục hạn chế này, đề tài xây dựng Động cơ giải thích rule-based độc lập (Explanation Engine) hoạt động độc lập với mạng nơ-ron lỗi.

Động cơ này hoạt động bằng cách đo lường trực tiếp các tập đặc trưng Heuristics trên văn bản thô, bao gồm các chỉ số như tỷ lệ chữ in hoa, mật độ dấu câu, độ dài tiêu đề. Dựa trên các tập luật (rules) được lập trình

sẵn, nó sẽ tự động biên dịch các chỉ số bất thường này thành những câu văn tiếng Việt thân thiện (ví dụ: "Lạm dụng dấu câu quá mức"). Các câu văn giải thích này được hiển thị đồng thời cùng với dự đoán xác suất, giúp minh bạch hóa lý do cảnh báo tin giả.

Sự tách biệt giữa mô hình dự đoán (PhoBERT) và mô hình giải thích (Heuristics) hỗ trợ tối ưu tốc độ suy luận và cải thiện khả năng diễn giải kết quả của hệ thống.

2.6. Kết luận chương

Chương 2 đã cung cấp khung lý thuyết, làm nền tảng khoa học cho toàn bộ quá trình thiết kế và phát triển hệ thống phát hiện tin giả. Thông qua việc phân tích chuyên sâu, đồ án đã phân tích kiến trúc mạng nơ-ron Transformer [6, 7], đặc biệt là sức mạnh nắm bắt ngữ nghĩa tiếng Việt của mô hình PhoBERT. Bên cạnh đó, việc khảo lược các công trình nghiên cứu tiên phong trong và ngoài nước đã khẳng định sự đúng đắn của phương pháp tiếp cận mạng đa tầng (PhoBERT Text-only + MLP) nhằm tối đa hóa độ chuẩn xác. Hơn nữa, việc nghiên cứu các cơ sở lý thuyết về Động cơ Giải thích độc lập (Rule-based Explanation) [5, 8] và hệ thống hóa toàn bộ công nghệ phát triển Web (FastAPI, Next.js, PyTorch) đã tạo ra một bức tranh toàn cảnh về mặt kỹ thuật. Những luận điểm lý thuyết và sự chuẩn bị về mặt công nghệ trong chương này chính là tiền đề quan trọng, định hướng trực tiếp cho các quyết định kiến trúc cốt lõi sẽ được thiết kế và mô hình hóa trong Chương 3 tiếp theo.

CHƯƠNG 3: PHƯƠNG PHÁP NGHIÊN CỨU VÀ THIẾT KẾ HỆ THỐNG

3.1. Phương pháp nghiên cứu

Để giải quyết bài toán phát hiện tin giả tiếng Việt, đồ án không áp dụng một thuật toán học máy đơn lẻ mà xây dựng một quy trình luận giải đa tầng (Multi-tier Analytical Pipeline). Phương pháp nghiên cứu được chia thành bốn mũi nhọn chiến lược, đi từ khâu thu thập dữ liệu thô, trích xuất đặc trưng, mô hình hóa suy luận, cho đến việc minh bạch hóa kết quả.

3.1.1. Phân tích Dữ liệu nghiên cứu (Dataset Analysis)

Mọi hệ thống Trí tuệ Nhân tạo đều được định hình bởi chất lượng của nguồn dữ liệu huấn luyện. Đồ án sử dụng bộ dữ liệu "**Vietnamese Fake News Dataset**" tổng hợp từ nhiều nguồn tin tức trên không gian mạng. Quá trình Khai phá và Phân tích Dữ liệu (Exploratory Data Analysis - EDA) đã chỉ ra các đặc tính học thuật quan trọng sau:

- **Số lượng mẫu và Phân bố nhãn (Label Distribution):** Bộ dữ liệu `full_dataset.csv` bao gồm tổng cộng **10.617 bản ghi (bài báo)**; sau lọc (`dataset_cleaner.py`) còn **10.609 mẫu** hợp lệ. Phân bố nhãn tương đối cân bằng (~53,6% tin thật / ~46,4% tin giả), giúp giảm rủi ro mô hình bị thiên lệch (class imbalance) trong quá trình huấn luyện.
- **Độ dài văn bản và Hiện tượng Mất cân bằng dữ liệu:** Các bài viết lừa đảo thường sử dụng từ ngữ thao túng kết hợp với cấu trúc câu giật gân, đe dọa. Qua thống kê độ dài trên 10.609 mẫu, dữ liệu không có sự chênh lệch quá lớn về số lượng từ giữa tin thật và tin giả, đảm bảo mô hình không học vẹt theo độ dài văn bản.
- **Cấu trúc trường thông tin:** Dữ liệu cung cấp các cột `title`, `content`, `url`, `source` phục vụ huấn luyện và crawl inference. Các tín hiệu hình thức (viết hoa, dấu câu) chỉ được dùng trong `ExplanationEngine` (XAI), không đưa vào vector đặc trưng mô hình.

3.1.2. Tiền xử lý và Làm sạch Dữ liệu (Data Cleaning)

Train và inference dùng chung hàm `preprocess_text` trong `text_utils.py` — tránh lệch pipeline. Các bước:

- **Chuẩn hóa chữ thường (lowercase):** Thống nhất biểu diễn văn bản.
- **Loại bỏ URL:** Regex xóa liên kết `http(s)://...` không mang ngữ nghĩa phân loại.
- **Chuẩn hóa khoảng trắng:** Gom nhiều khoảng trắng thành một, loại bỏ khoảng trắng thừa.
- **Tách từ PyVi:** `ViTokenizer` nối từ ghép bằng dấu gạch dưới (ví dụ: `tin_giả`, `y_tê`) trước khi đưa vào PhoBERT.

Lưu ý: Module helper `TextCleaner` có thêm xóa HTML/teencode nhưng **không** nằm trên luồng `preprocess_text` chính. Crawler HTML chỉ dùng khi người dùng nhập URL (BeautifulSoup trích title/content).

3.1.3. Phân chia Dữ liệu Huấn luyện (Data Split)

Do quy mô dữ liệu khổng lồ, tập dữ liệu được phân chia theo tỷ lệ **70% (Train) - 30% (Test)** để phục vụ cho các vòng đời của mô hình AI. Đặc biệt, kỹ thuật phân tầng (Stratified Split) được áp dụng nghiêm ngặt trong hàm `train_test_split` của thư viện Scikit-learn để đảm bảo tỷ lệ phân bố nhãn luôn được giữ nguyên. Ngoài ra, để tăng tính thuyết phục học thuật, đồ án còn áp dụng thêm kỹ thuật **Kiểm định chéo 5-Fold (Cross-validation)** trên tập Train. Việc tạo ra tập Test hoàn toàn biệt lập (chiếm tới 30% dữ liệu) là cơ chế phòng vệ cốt lõi để loại trừ hiện tượng Rò rỉ dữ liệu (Data Leakage), đảm bảo độ tin cậy của các chỉ số hiệu năng.

3.1.4. Phương pháp Trích xuất biểu diễn ngữ nghĩa bằng PhoBERT

Đồ án sử dụng mô hình ngôn ngữ tiền huấn luyện PhoBERT (dựa trên kiến trúc Transformer) làm thành phần cốt lõi để trích xuất đặc trưng văn bản. Văn bản tiếng Việt sau khi làm sạch sẽ được đưa qua bộ Tokenizer của PhoBERT để chuyển đổi thành chuỗi các sub-words. Hệ thống trích xuất vector nhúng (embedding) ở vị trí token đặc biệt [CLS] tại lớp ẩn cuối cùng (last hidden state). Vector 768 chiều này chứa đựng biểu diễn ngữ nghĩa sâu sắc của toàn bộ đoạn văn bản, bao gồm cả cấu trúc ngữ pháp và sự tương quan ngữ cảnh đa chiều giữa các từ. Đây là nguồn dữ liệu duy nhất và giàu thông tin nhất được sử dụng làm đầu vào cho bộ phân loại Học máy lỗi, đảm bảo mô hình không bị nhiễu bởi các yếu tố thủ công.

3.1.5. Phương pháp Mô hình hóa mạng đa tầng (PhoBERT + MLP)

Trái tim của hệ thống là phương pháp **PhoBERT Text-only + MLP**. Bộ phân loại được triển khai dựa trên thuật toán `MLPClassifier` của thư viện `scikit-learn` (cấu hình mạng: lớp ẩn 128, 64; hàm kích hoạt ReLU; thuật toán tối ưu Adam; `alpha=0.1`; `early_stopping=True`). Vector ngữ nghĩa 768 chiều từ PhoBERT sau khi được trích xuất sẽ được chuẩn hóa thông qua `StandardScaler` trước khi đưa vào mạng nơ-ron. Lưu ý rằng bộ tín hiệu hình thức (Heuristics) không được nối trực tiếp vào pipeline phân loại, mà chỉ được sử dụng độc lập trong `ExplanationEngine` ở bước suy luận cuối cùng.

3.1.6. Phương pháp Đánh giá và Giải thích (Explainability Integration)

Về mặt đánh giá, hệ thống áp dụng phương pháp Validation phân tầng (Stratified K-Fold) và sử dụng chỉ số F1-Score làm thước đo cốt lõi. Để tích hợp **ExplanationEngine** rule-based: sau khi MLP trả xác suất, module quét văn bản gốc (viết hoa, dấu câu giật gân, độ dài tiêu đề...) và sinh câu giải thích tiếng Việt — không dùng SHAP/LIME hay gradient ngược trên trọng số MLP.

3.2. Công nghệ triển khai hệ thống phần mềm

Phần này trình bày các công nghệ kỹ thuật phần mềm được lựa chọn để hiện thực hóa mô hình AI thành một ứng dụng hoàn chỉnh.

3.3. Phân tích yêu cầu

***Lưu ý của tác giả:** Do tính chất và phạm vi của một đồ án Trí tuệ Nhân tạo, phần phân tích yêu cầu dưới đây chỉ được thiết kế ở mức độ cơ bản nhất để phục vụ cho việc kiểm thử mô hình Học máy, không đặt nặng tiêu chuẩn kỹ nghệ phần mềm.

3.3.1. Yêu cầu chức năng

Dựa trên mục tiêu nghiên cứu và bối cảnh sử dụng thực tế, hệ thống phát hiện tin giả được thiết kế với các yêu cầu chức năng cốt lõi (Bảng 3.1), chia làm 5 phân hệ (Module) chi tiết như sau:

Bảng 3.1. Yêu cầu chức năng hệ thống ShieldAI

Mã	Yêu cầu	Mô tả
FR-01	Nhập văn bản	Người dùng dán tiêu đề và nội dung bài viết
FR-02	Nhập URL	Crawl và trích xuất title/content từ báo điện tử
FR-03	Tiền xử lý	<code>preprocess_text</code> : lowercase, xóa URL, PyVi tokenize (luồng train/inference)
FR-04	Trích đặc trưng	PhoBERT embedding [CLS] 768 chiều
FR-05	Phân loại	Xác suất tin giả; verdict 3 mức (<35% / 35-74% / ≥75%)
FR-06	Giải thích	ExplanationEngine rule-based, JSON tiếng Việt
FR-07	Lịch sử & xác thực	JWT; lưu/xem lịch sử phân tích
FR-08	API REST	<code>/api/analyze</code> , <code>/api/health</code> , <code>/api/auth/*</code>

1. Phân hệ Quản lý Tài khoản (User Management)

- **Đăng ký và Đăng nhập:** Hệ thống cho phép người dùng tạo tài khoản mới và đăng nhập thông qua cơ chế xác thực an toàn.
- **Quản lý Phiên (Session):** Cấp phát mã thông báo (Token JWT) để duy trì phiên làm việc của người dùng mà không cần lưu trữ mật khẩu trực tiếp.

2. Phân hệ Thu thập & Tiền xử lý Dữ liệu (Data Scraping & Preprocessing)

- **Đầu vào Đa dạng:** Chức năng cho phép người dùng kiểm tra thông tin bằng 2 cách: (1) Nhập trực tiếp đoạn văn bản thô, hoặc (2) Cung cấp đường dẫn mạng (URL) của bài báo cần kiểm chứng.
- **Trích xuất tự động (Web Scraping):** Khi người dùng nhập URL, hệ thống phải tự động truy cập, bóc tách chính xác thành phần Tiêu đề (Title) và Nội dung bài báo (Content). Chức năng này phải có bộ lọc thông minh để loại bỏ các mã HTML không cần thiết như menu điều hướng, quảng cáo, hay thẻ script.
- **Chuẩn hóa Ngôn ngữ:** Áp dụng `preprocess_text` : lowercase → xóa URL → chuẩn hóa khoảng trắng → PyVi tokenize (ví dụ: "đại học" → "đại_học"). Cùng pipeline cho train và inference.

3. Phân hệ Lỗi: Suy luận bằng PhoBERT và Mạng nơ-ron đa tầng (Core Inference)

- **Trích xuất embedding:** Hệ thống trích vector [CLS] 768 chiều từ PhoBERT, chuẩn hóa bằng `StandardScaler` và phân loại qua MLP (scikit-learn).
- **Suy luận tuần tự:** Luồng: thu thập văn bản → `preprocess_text` → PhoBERT embedding → MLP → verdict 3 mức → `ExplanationEngine` (rule-based, chạy sau phân loại).
- **Phân loại và Cảnh báo (Verdict):** Đưa ra phán quyết cuối cùng dưới dạng 3 mức độ (Tin thật, Đáng ngờ, Tin giả) thay vì nhị phân cứng nhắc. Đánh giá này dựa trên chỉ số xác suất rủi ro (Confidence Score) từ 0% đến 100% do AI cung cấp.

4. Phân hệ Trí tuệ Nhân tạo Minh bạch (Heuristic Explanation Engine - Giải thích Heuristic)

- **Phân tích tín hiệu hình thức:** `ExplanationEngine` đo các chỉ số rule-based (viết hoa, dấu câu giắt gàn, độ dài tiêu đề...) trên văn bản gốc và sinh câu giải thích — không phân tích ngược trọng số nội bộ MLP.
- **Sinh Báo cáo Trực quan:** Trả về báo cáo dưới dạng JSON để Frontend hiển thị thành các thanh trạng thái (Progress Bars), giúp người dùng đọc hiểu dễ dàng tại sao bài báo lại bị dán nhãn lừa đảo.

5. Phân hệ Quản lý Lịch sử (History Tracking)

- **Lưu trữ Lướt quét:** Tự động lưu lại toàn bộ các văn bản hoặc URL mà người dùng đã kiểm chứng vào cơ sở dữ liệu.
- **Truy xuất Báo cáo cũ:** Cho phép người dùng xem lại kết quả phân loại và bảng báo cáo Heuristic của các bài báo cũ mà không cần phải bắt hệ thống phân tích lại từ đầu.

3.3.2. Yêu cầu phi chức năng

Để đảm bảo hệ thống không chỉ là một mô hình học máy chạy trên môi trường cục bộ (Local) mà có thể sẵn sàng phục vụ người dùng cuối trong môi trường sản xuất (Production), các yêu cầu phi chức năng (Non-Functional Requirements) được định nghĩa khắt khe qua 5 khía cạnh sau:

1. Hiệu suất và Độ trễ (Performance & Latency)

- **Giới hạn thời gian phản hồi:** Đối với phương thức nhập URL, hệ thống hướng tới việc tối ưu hóa thời gian xử lý, đảm bảo độ trễ ở mức chấp nhận được trong môi trường thử nghiệm.
- **Tải trọng đồng thời (Concurrency):** Backend (FastAPI) áp dụng cấu hình bất đồng bộ (Asynchronous) nhằm cải thiện khả năng chịu tải so với các framework đồng bộ truyền thống.

2. Bảo mật và An toàn hệ thống (Security)

- **Bảo mật phiên làm việc:** Kiến trúc stateless với **JSON Web Token (JWT)**. API trả JSON thống nhất: HTTP **200** kèm trường `status` (`success` / `error`) và `message` — kể cả lỗi đăng nhập, thiếu JWT hay nội dung trống (frontend đọc `status` , không dựa mã 401/403).
- **Mã hóa dữ liệu nhạy cảm:** Toàn bộ mật khẩu của người dùng khi đăng ký phải được băm một chiều bằng thuật toán **Bcrypt** trước khi ghi vào cơ sở dữ liệu (SQLite/PostgreSQL).
- **Chống lạm dụng API:** Hệ thống phải tích hợp cơ chế hạn chế tốc độ (Rate Limiting) trên các endpoint phân tích để chống lại các cuộc tấn công từ chối dịch vụ (DDoS) hoặc việc bot spam URL liên tục.

3. Khả năng mở rộng và Tính khả dụng (Scalability & Availability)

- **Kiến trúc Tách rời (Decoupled Architecture):** Việc tách biệt Frontend (Next.js) và Backend (FastAPI) qua giao tiếp RESTful API đảm bảo rằng việc cập nhật giao diện không làm ảnh hưởng đến lõi mô hình AI và ngược lại.
- **Sẵn sàng đóng gói (Docker-ready):** Cấu trúc mã nguồn phải sẵn sàng cho việc đóng gói vào các Container (Docker), tạo tiền đề cho việc triển khai linh hoạt lên đám mây (Cloud) như AWS hay Heroku.

4. Trải nghiệm người dùng (Usability & UX)

- **Khả năng tương thích chéo (Responsive Design):** Giao diện Web phải hiển thị hiệu quả trên cả trình duyệt máy tính (Desktop) lẫn thiết bị di động (Mobile). Các biểu đồ phân tích phải được tối ưu kích thước để không bị tràn màn hình.
- **Cảnh báo lỗi thân thiện (Graceful Error Handling):** Khi cào một URL thất bại (do trang báo chặn bot hoặc URL chết), hệ thống không được crash mà phải trả về một thông báo lỗi (Toast message) rõ ràng, dễ hiểu cho người dùng.

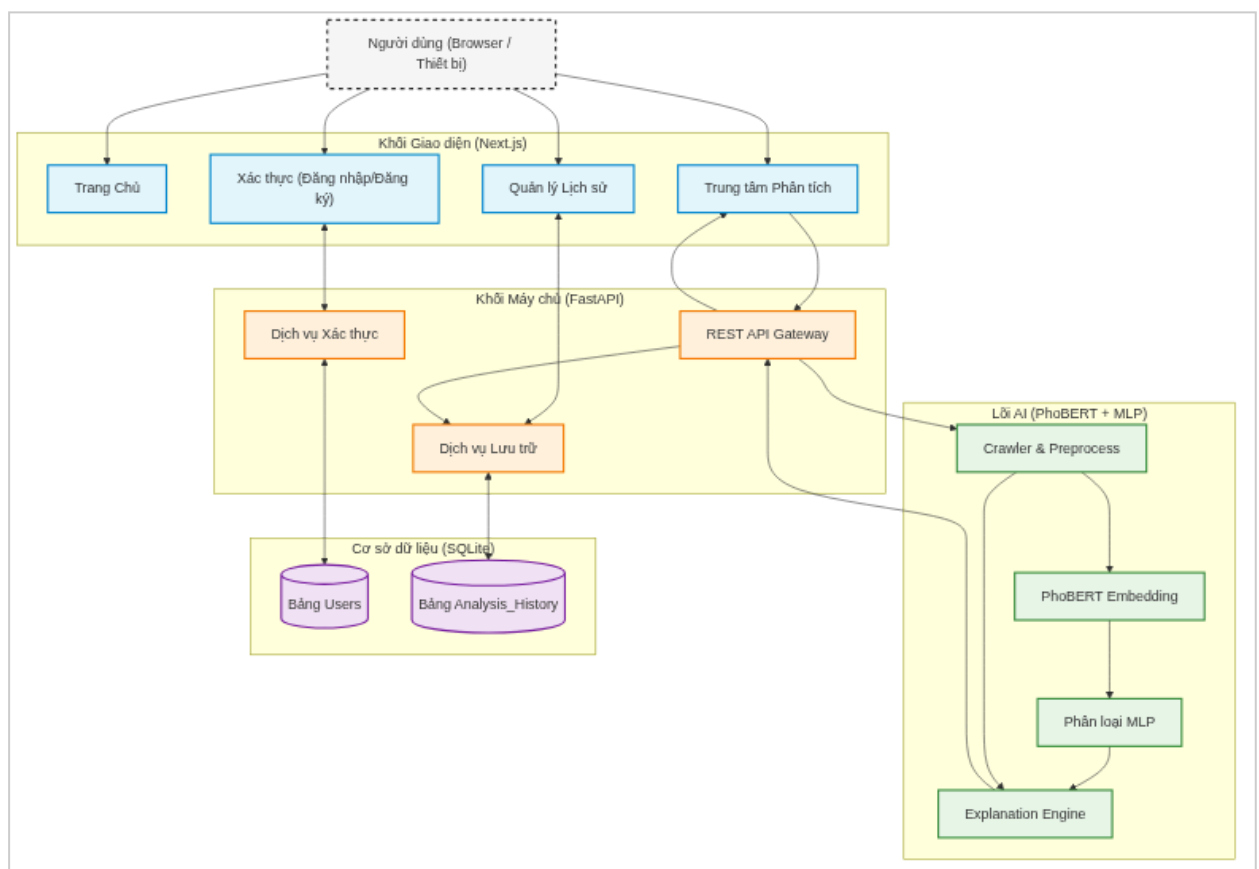
5. Khả năng bảo trì và Nâng cấp mô hình (Maintainability & MLOps)

- **Cấu trúc thư mục chuẩn:** Mã nguồn Python tuân cấu trúc tách module (`models/` , `api/` , `text_utils.py` , ...); các hàm lỗi có Docstring đầy đủ.
- **Tách biệt artifact:** PhoBERT tải qua Hugging Face; MLP + StandardScaler lưu `.joblib` trong `models/` , không nhúng vào mã nguồn — để retrain mà không đổi logic Backend.

3.4. Thiết kế hệ thống

3.4.1. Kiến trúc hệ thống

Cấu trúc hệ thống (System Architecture) là cách các thành phần của một hệ thống được tổ chức, sắp xếp và liên kết với nhau để hệ thống hoạt động và đạt được mục tiêu đề ra. Dựa trên định nghĩa cốt lõi đó và các yêu cầu đã phân tích, dự án ShieldAI được thiết kế theo kiến trúc phân tán **Client-Server (Front-end & Back-end Separation)** với mô hình 3 tầng (3-Tier) độc lập. Việc tách biệt các tầng giúp tối đa hóa khả năng bảo trì mã nguồn, tăng tốc độ truy vấn cơ sở dữ liệu và tối ưu hóa tài nguyên phần cứng khi xử lý các mô hình học sâu. Cụ thể, cách các thành phần của hệ thống được tổ chức và liên kết với nhau được minh họa trực quan trong sơ đồ cấu trúc dưới đây:



Hình 3.1: Sơ đồ Kiến trúc Hệ thống Phân tán

Như minh họa trong sơ đồ trên, thay vì gộp chung mọi logic xử lý vào một khối nguyên khối (Monolith Architecture), đồ án đã áp dụng kiến trúc phân tán **3 tầng (3-Tier Micro-Architecture) độc lập** (cùng với 1 tầng lưu trữ). Việc chia tách này không chỉ giúp quản lý mã nguồn dễ dàng mà còn đảm bảo tính mở rộng (Scalability), tính tái sử dụng và bảo mật (Security) cho hệ thống trong môi trường thực tế. Sự giao tiếp giữa các tầng được kiểm soát chặt chẽ thông qua các chuẩn HTTP và dữ liệu JSON. Cụ thể, các tầng được thiết kế chi tiết như sau:

Tầng 1: Tầng Giao diện Người dùng (Presentation / Client Tier)

Tầng Giao diện đóng vai trò là điểm chạm đầu tiên (touchpoint) giữa người dùng và hệ thống. Được xây dựng hoàn toàn bằng **Next.js 14** (một React Framework hiện đại) kết hợp cùng **Tailwind CSS** và thư viện chuyển động **Framer Motion**, tầng này đảm bảo trải nghiệm đa nền tảng (Responsive Design) và tương tác ổn định từ màn hình máy tính đến thiết bị di động. Điểm cốt lõi trong thiết kế của Tầng Giao diện là nó hoạt động độc lập (agnostic) với các thuật toán Trí tuệ Nhân tạo phức tạp bên dưới. Nhiệm vụ của nó chỉ là tiếp nhận yêu cầu đầu vào từ người dùng (một đoạn văn bản nghi ngờ hoặc một đường dẫn URL bài báo), đóng gói thành các truy vấn chuẩn HTTP RESTful, và gửi đi. Khi nhận được kết quả trả về từ máy chủ, module **Trực quan hóa Dữ liệu** sẽ sử dụng các công cụ vẽ biểu đồ linh hoạt để kết xuất các thông số đánh giá rõ ràng (ví dụ: thanh tiến trình hiển thị 99% giả mạo, biểu đồ radar cho các chỉ số cảm xúc, hoặc bôi đỏ các từ ngữ giật gân). Cơ chế này giúp người dùng phổ thông, dù không có chuyên môn về máy học, vẫn có thể dễ dàng nắm bắt được kết quả phân loại.

Tầng 2: Tầng Xử lý Nghiệp vụ (Application / Backend Tier)

Đây là "trái tim điều phối" của toàn bộ nền tảng phần mềm, được triển khai bằng framework **FastAPI**. Các endpoint trả JSON với envelope `{"status": "success"|"error", "message": "...", ...}`; mã HTTP thường là **200** (kể cả lỗi nghiệp vụ như sai mật khẩu hoặc chưa đăng nhập). Ba phân hệ chính: (1) **JWT Auth**; (2) **Web Scraper** (BeautifulSoup khi nhập URL); (3) **Router** điều phối sang Tầng AI.

Tầng 3: Tầng Suy luận Trí tuệ Nhân tạo (AI Inference Tier)

Tầng này chứa pipeline suy luận ShieldAI: **PhoBERT** (PyTorch/Transformers) trích embedding [CLS] 768 chiều; **StandardScaler + MLPClassifier** (scikit-learn, file `.joblib`) tính xác suất tin giả; `verdict.py` quy đổi 3 mức. Sau đó **ExplanationEngine** (rule-based) quét văn bản gốc (in hoa, dấu câu giật gân...) để sinh JSON giải thích tiếng Việt — **không** tham gia phân loại.

Tầng 4: Tầng Lưu trữ Dữ liệu (Data Storage Tier)

Mặc dù là tầng thấp nhất, Tầng Lưu trữ lại đóng vai trò tối quan trọng trong việc bảo toàn tính bền vững của nền tảng. Hệ thống sử dụng **SQLite** kết nối qua mô hình ORM của thư viện **SQLAlchemy**. Mọi kết quả phân tích JSON chi tiết từ Tầng AI, thông tin tài khoản người dùng, và lịch sử hoạt động đều được lưu vết vĩnh viễn tại đây. Cơ chế này mang lại ưu điểm đáng kể: cho phép người dùng có thể tra cứu ngay lập tức các bài báo họ đã quét trong quá khứ thông qua bảng điều khiển cá nhân (Dashboard), mà không cần phải bắt các cụm GPU/CPU của hệ thống AI phải vất vả chạy lại các phép toán nặng nề cho một dữ liệu cũ.

3.4.2. Thiết kế cơ sở dữ liệu và Mô hình Thực thể - Mối quan hệ (ERD)

Để đảm bảo tính bền vững và nhất quán của luồng dữ liệu, hệ thống sử dụng hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) **SQLite** kết hợp cùng công nghệ ánh xạ đối tượng **SQLAlchemy ORM (Object-Relational Mapping)**. Việc lựa chọn SQLite được xem là một quyết định kiến trúc chiến lược: nó cung cấp một giải pháp lưu trữ tối ưu (lightweight), không đòi hỏi cấu hình máy chủ cơ sở dữ liệu phức tạp, nhưng vẫn tuân thủ đầy đủ các tiêu chuẩn ACID (Atomicity, Consistency, Isolation, Durability) để đảm bảo an toàn dữ liệu. Thay vì viết các câu lệnh SQL thô (Raw SQL) dễ gây ra lỗ hổng bảo mật SQL Injection, SQLAlchemy ORM cho phép thao tác với dữ liệu dưới dạng các đối tượng (Objects) trong Python, giúp mã nguồn dễ đọc và bảo trì.

Mô hình Thực thể - Mối quan hệ (Entity-Relationship Diagram - ERD) của hệ thống được thiết kế theo hướng tinh gọn, tập trung chủ yếu vào việc giải quyết bài toán cốt lõi: lưu trữ định danh người dùng, vết tích của quá trình phân tích AI, và thu thập phản hồi của người dùng để liên tục cải thiện mô hình. Hệ thống bao gồm ba thực thể chính là bảng **USERS** (Người dùng), bảng **ANALYSIS_HISTORY** (Lịch sử phân tích) và bảng **FEEDBACKS** (Phản hồi).

![Sơ đồ Thực thể - Mối quan hệ ERD](image/erd.png)
Sơ đồ Thực thể - Mối quan hệ (ERD)

Chi tiết về các bảng và cấu trúc trường (Fields) được phân tích như sau:

- **Bảng USERS (Quản lý Tài khoản):** Chịu trách nhiệm lưu trữ thông tin cá nhân. Trường `email` và `username` được đánh chỉ mục độc nhất (Unique Index) để làm định danh đăng nhập. Mật khẩu không được lưu dưới dạng văn bản thô, mà sử dụng thuật toán băm một chiều để tạo ra trường `hashed_password` nhằm ngăn chặn rò rỉ dữ liệu.
- **Bảng ANALYSIS_HISTORY (Lịch sử Phân tích):** Đây là quyển "Sổ cái" ghi lại chi tiết hoạt động của Động cơ AI. Bên cạnh các dữ liệu đầu vào như `input_mode` (URL/Text), `title`, `content`, bảng còn lưu trữ chi tiết phán quyết của AI thông qua `fake_prob` (xác suất) và `verdict` (3 ngưỡng cảnh báo). Đặc biệt, trường `explanation_json` và `meta_json` cho phép linh hoạt lưu trữ báo cáo Heuristics bán cấu trúc (NoSQL-like) ngay bên trong CSDL quan hệ.
- **Bảng FEEDBACKS (Quản lý Phản hồi):** Đóng vai trò quan trọng trong chiến lược MLOps (Machine Learning Operations). Bảng cho phép người dùng đánh giá tính chính xác của phán quyết AI (`is_correct`) và để lại bình luận (`comment`). Dữ liệu này là nguồn tài nguyên quý giá để nhóm nghiên cứu tái huấn luyện (Re-train) mô hình trong tương lai.

Bảng 3.2. Từ điển Dữ liệu (Data Dictionary) cho các Thực thể

Thực thể	Trường dữ liệu	Kiểu dữ liệu	Mô tả chi tiết
USERS	id	Integer	Khóa chính (PK), tự động tăng.
USERS	email	String(255)	Email đăng nhập (Unique, Index).
USERS	username	String(64)	Tên người dùng hiển thị (Unique, Index).
USERS	hashed_password	String(255)	Mật khẩu đã mã hóa.
USERS	created_at	DateTime	Thời điểm khởi tạo tài khoản.
ANALYSIS_HISTORY	id	Integer	Khóa chính (PK), tự động tăng.
ANALYSIS_HISTORY	user_id	Integer	Khóa ngoại (FK) trỏ đến bảng USERS.
ANALYSIS_HISTORY	input_mode	String(10)	Loại đầu vào: "text" hoặc "url".
ANALYSIS_HISTORY	title	String(500)	Tiêu đề bài báo (nếu có).
ANALYSIS_HISTORY	content	Text	Nội dung bài báo hoặc đoạn văn bản.
ANALYSIS_HISTORY	source_url	String(2048)	URL gốc của bài báo (nếu input_mode="url").
ANALYSIS_HISTORY	source_domain	String(255)	Tên miền gốc của URL (nếu có).
ANALYSIS_HISTORY	result_label	String(64)	Nhãn dự đoán cuối cùng.
ANALYSIS_HISTORY	fake_prob	Float	Xác suất giả mạo từ AI (0.0 đến 1.0).
ANALYSIS_HISTORY	verdict	String(32)	Ngưỡng cảnh báo (Tin Thật/Đáng ngờ/Tin Giả).
ANALYSIS_HISTORY	explanation_json	Text	Báo cáo giải thích minh bạch từ Heuristics.
ANALYSIS_HISTORY	meta_json	Text	Thông tin siêu dữ liệu mở rộng.
ANALYSIS_HISTORY	created_at	DateTime	Thời gian phân tích.
FEEDBACKS	id	Integer	Khóa chính (PK), tự động tăng.
FEEDBACKS	history_id	Integer	Khóa ngoại (FK) trỏ 1-1 về bảng ANALYSIS_HISTORY.
FEEDBACKS	user_id	Integer	Khóa ngoại (FK) trỏ về bảng USERS.
FEEDBACKS	is_correct	Boolean	Đánh giá dự đoán có chính xác không (Thumbs up/down).

FEEDBACKS	comment	Text	Nhận xét chi tiết từ người dùng (nếu có).
-----------	---------	------	---

Mối quan hệ giữa các thực thể này là mối quan hệ **1-N (Một - Nhiều)** giữa người dùng với lịch sử phân tích, và quan hệ **1-1 (Một - Một)** giữa bản phân tích và một phản hồi duy nhất. Việc ứng dụng ORM SQLAlchemy giúp liên kết tường minh thông qua các khóa ngoại (Foreign Keys). Khái niệm này phản ánh luồng nghiệp vụ thực tế: Một người dùng quét nhiều bài báo, họ đánh giá các báo cáo này, và toàn bộ dữ liệu trở thành quy mô tri thức mới khổng lồ để cải thiện AI trong tương lai.

3.4.3. Thiết kế giao diện (UI/UX Design) và Trải nghiệm Người dùng

Với mục tiêu đưa công nghệ Trí tuệ Nhân tạo phức tạp đến gần hơn với người dùng phổ thông, Giao diện người dùng (User Interface - UI) và Trải nghiệm người dùng (User Experience - UX) của hệ thống được thiết kế theo triết lý **Tối giản và Hiện đại (Modern Minimalism)**. Toàn bộ nền tảng Frontend được xây dựng bằng Next.js kết hợp với bộ công cụ **Tailwind CSS**. Việc ứng dụng Tailwind giúp hệ thống duy trì được một cấu trúc mã màu (Color Palette) nhất quán, áp dụng chuẩn thiết kế bo góc (border-radius mềm mại), hiệu ứng kính mờ (Glassmorphism), và đặc biệt là khả năng tương thích hiển thị hiệu quả trên mọi kích thước màn hình (Fully Responsive) từ màn hình Desktop rộng lớn cho đến điện thoại di động. Tổng thể giao diện hệ thống bao gồm ba phân hệ màn hình cốt lõi, mỗi phân hệ được thiết kế để giải quyết một điểm chạm cụ thể của người dùng:

1. Màn hình Cổng thông tin và Phân tích (Home & Analysis Portal)

Đây là màn hình đầu tiên khi người dùng truy cập vào ứng dụng. Thiết kế tập trung sự chú ý của người dùng vào một **Thanh tìm kiếm (Search Bar)** kích thước lớn đặt ở chính giữa không gian làm việc. Thanh tìm kiếm này được thiết kế thông minh với tính năng đa định dạng đầu vào (Multi-input format): người dùng có thể dán (paste) trực tiếp một đường dẫn URL của bài báo mạng, hoặc gõ một đoạn văn bản thô bất kỳ vào khung soạn thảo. Ngay phía dưới là nút "Phân tích nội dung" được tích hợp các hiệu ứng vi mô (micro-interactions) phản hồi xúc giác khi rê chuột (hover). Trong quá trình chờ máy chủ chạy các phép toán AI, màn hình sẽ hiển thị các hiệu ứng tải (Loading animations) dạng sóng mượt mà, nhằm cải thiện trải nghiệm người dùng (UX optimization).

2. Màn hình Báo cáo Giám định (Result & Giải thích Heuristic Dashboard)

Màn hình báo cáo kết quả chính là nơi trình diễn toàn bộ khả năng của công nghệ Trí tuệ Nhân tạo Minh bạch (Heuristic Explanation). Khác với các hệ thống truyền thống chỉ hiện một dòng chữ khô khan, màn hình này áp dụng nguyên lý thiết kế **Thị giác hóa mức độ rủi ro (Visual Risk Assessment)** thông qua mã màu: **Màu Đỏ/Cam** sẽ chớp cảnh báo cho các tin giả độc hại và **Màu Xanh lá cây** sẽ đại diện cho nguồn thông tin an toàn, đáng tin cậy. Giao diện được chia bố cục làm hai vùng chính (Grid Layout):

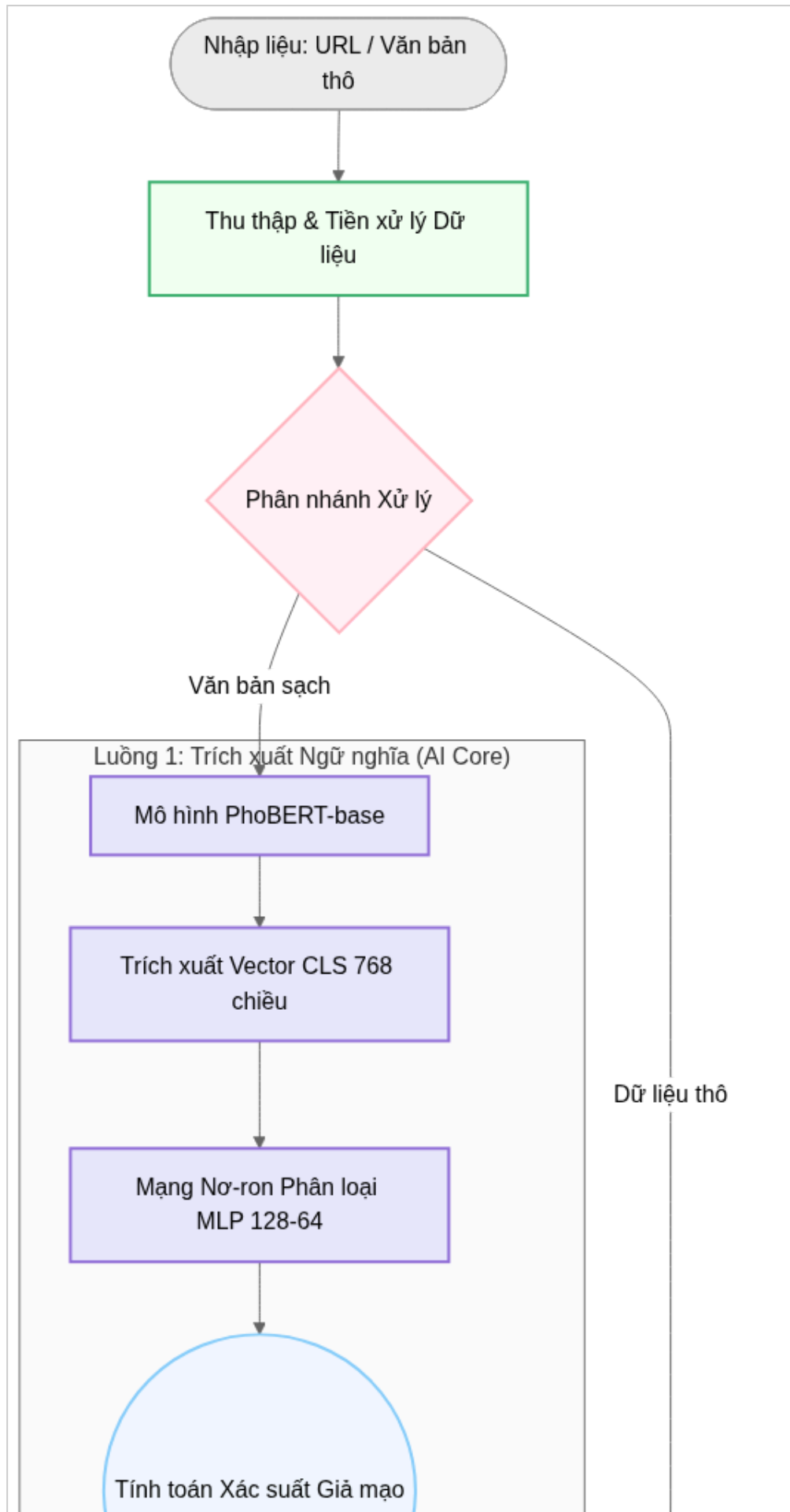
- **Vùng Chỉ số Tổng quan:** Hiển thị một vòng tròn đo lường tỷ lệ phần trăm (Confidence Score) cực lớn, giúp người dùng nắm bắt ngay kết luận cuối cùng (ví dụ: "Bài báo này 94% là Giả mạo") chỉ trong vòng 2 giây đầu tiên lướt nhìn.
- **Vùng Phân tích Chuyên sâu (Giải thích Heuristic Metrics):** Khu vực này hiển thị các thanh đo (Progress bars) bóc tách chi tiết lý do tại sao văn bản lại bị AI đánh dấu như vậy. Các chỉ số như "Tỷ lệ lạm dụng viết hoa", "Mật độ sử dụng dấu chấm than", "Số lượng từ lóng", hay "Chỉ số cảm xúc tiêu cực" được trực quan hóa sinh động. Nhờ tận dụng cấu trúc Component của React, các thanh đo thị này được thiết lập hiệu ứng chạy mượt mà từ 0% lên đến mức thực tế, cung cấp cho người dùng thông tin chi tiết về các yếu tố ảnh hưởng đến kết quả phân loại.

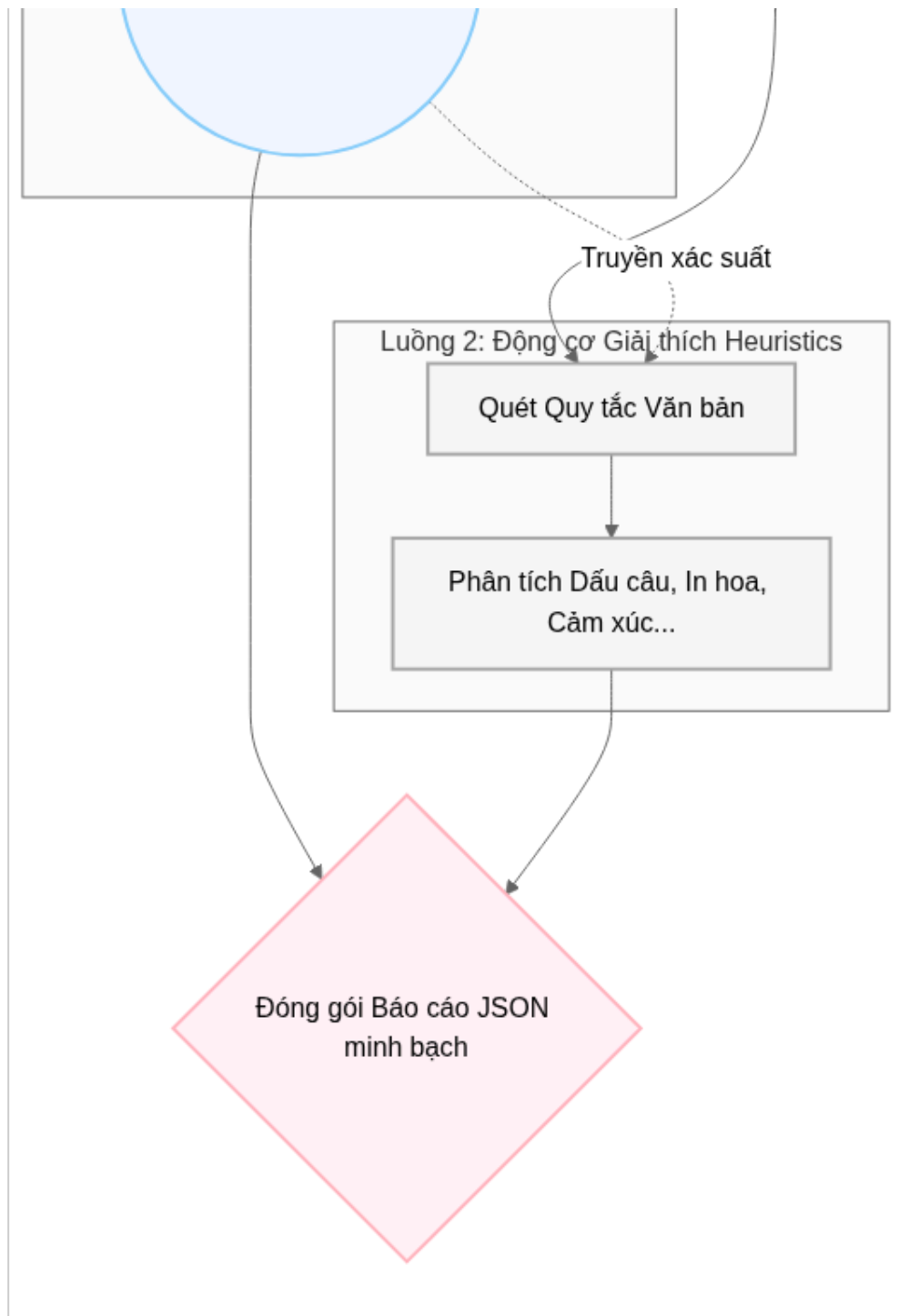
3. Màn hình Quản lý Lịch sử (Personal History View)

Để tăng tính gắn kết cá nhân hóa, hệ thống cung cấp một bảng điều khiển lưu trữ toàn bộ các phiên phân tích trong quá khứ của người dùng. Màn hình này áp dụng thiết kế dạng danh sách thẻ động (Dynamic Cards), cho phép người dùng lướt nhanh qua các tiêu đề bài báo đã quét, thời gian thực hiện thao tác và đóng dấu nhãn Thật/Giả một cách rõ ràng. Điểm tối ưu về mặt UI/UX ở đây là việc triển khai tính năng phân trang (Pagination) ngay tại phía máy chủ kết hợp bộ lọc động, giúp giao diện Frontend có thể tải và cuộn qua hàng trăm bản ghi lịch sử một cách mượt mà, tối ưu hóa mức sử dụng bộ nhớ (RAM) và hạn chế tình trạng giật lag của trình duyệt.

3.4.4. Thiết kế thuật toán (Thuật toán PhoBERT + MLP - PhoBERT + MLP)

Đồ án triển khai thuật toán **PhoBERT Text-only + MLP**. Phân loại dựa trên embedding ngữ nghĩa 768 chiều; giải thích rule-based chạy **sau** bước phân loại, không ghép (concatenate) đặc trưng hình thức vào MLP. Lưu đồ dưới đây mô tả luồng từ văn bản đầu vào đến verdict và báo cáo giải thích.





Theo lưu đồ trên, thuật toán chạy theo pipeline tuần tự: tiền xử lý → embedding → phân loại → giải thích. Chi tiết các bước:

- **Bước 1: Tiền xử lý (Preprocessing):** Áp dụng `preprocess_text` : lowercase → xóa URL → chuẩn hóa khoảng trắng → PyVi tokenize. Cùng logic cho train và inference.
- **Bước 2: Giải mã Ngữ nghĩa (PhoBERT):** Văn bản sạch được đưa qua bộ Tokenizer của PhoBERT để băm nhỏ thành các mảnh từ vựng (Sub-words). Sau đó, chuỗi token này đi xuyên qua 12 lớp Mạng Tự chú ý (Self-Attention) của kiến trúc Transformer [6, 7]. Kết quả trả về là một siêu vector đặc trưng đại diện toàn cục cho ngữ cảnh của câu (thường gọi là [CLS] token vector), có kích thước cực lớn lên đến **768 chiều**. Vector này nắm giữ toàn bộ "ý nghĩa sâu xa" và "giọng điệu mỉa mai" ẩn giấu trong bài báo.
- **Bước 3: Phân loại (StandardScaler + MLPClassifier):** Vector 768 chiều được `StandardScaler` chuẩn hóa, đưa vào `MLPClassifier` (128, 64). `predict_proba` trả xác suất tin giả; `verdict.py` quy đổi 3 mức: <35% tin thật, 35-74% đáng ngờ, ≥75% tin giả.

- **Bước 4: Giải thích (ExplanationEngine):** Module rule-based quét văn bản gốc (viết hoa, dấu chấm than, văn phong giật gân), kết hợp xác suất từ MLP để sinh JSON giải thích tiếng Việt.

Để minh họa logic lập trình, luồng suy luận được tổng hợp thành đoạn Mã giả (Pseudocode) sau:

```

ALGORITHM: PhoBERT Text-only MLP Inference & Rule-based Explanation
INPUT:
    T: Văn bản gốc hoặc URL bài viết
OUTPUT:
    Verdict (Tin Thật/Đáng ngờ/Tin Giả), Confidence_Score, Explanation_Report

BEGIN
    // Bước 1: Tiền xử lý (preprocess_text)
    IF T is URL THEN
        T = CRAWL_AND_EXTRACT(T)    // BeautifulSoup: title + content
    END IF
    T_clean = PREPROCESS_TEXT(T) // lowercase, xóa URL, whitespace, PyVi

    // Bước 2: Trích xuất embedding PhoBERT (PyTorch/Transformers)
    Tokens = PHOBERT_TOKENIZER(T_clean)
    V_phobert = PHOBERT_MODEL(Tokens).extract_CLS() // Vector [CLS] 768 chiều
    V_scaled = STANDARD_SCALER.transform(V_phobert)

    // Bước 3: Phân loại (sklearn MLPClassifier)
    Confidence_Score = MLP_CLASSIFIER.predict_proba(V_scaled)[1] // xác suất tin giả

    // Bước 4: Verdict 3 mức (verdict.py)
    Verdict = MAP_TO_VERDICT(Confidence_Score) // <0.35 / 0.35–0.74 / ≥0.75

    // Bước 5: Giải thích rule-based (ExplanationEngine)
    Explanation_Report = EXPLANATION_ENGINE(T, Confidence_Score, Verdict)
    // Quét văn bản gốc: viết hoa, dấu câu, văn phong giật gân → JSON tiếng Việt

    RETURN Verdict, Confidence_Score, Explanation_Report
END

```

3.5. Kết luận chương

Chương 3 đóng vai trò là bản lề định hình toàn bộ khung xương kỹ thuật của đồ án, chuyển tiếp từ các lý thuyết nền tảng (Chương 2) sang một bản thiết kế hệ thống phần mềm hoàn chỉnh và sẵn sàng để lập trình thực tế. Nhìn lại toàn bộ chương, đồ án đã giải quyết ba bài toán cốt lõi trong việc xây dựng một hệ thống Trí tuệ Nhân tạo ứng dụng thực tiễn:

Thứ nhất, về mặt Kiến trúc Phần mềm: Đồ án thiết kế hệ thống theo kiến trúc **Micro-Architecture 3 tầng (3-Tier) độc lập**. Việc chia tách Tầng Giao diện (Next.js), Tầng Nghiệp vụ (FastAPI) và Tầng Suy luận AI (PhoBERT + MLP) đảm bảo tính bảo mật, khả năng mở rộng và chịu tải trong môi trường mạng thực tế.

Thứ hai, về mặt Thiết kế Dữ liệu và Trải nghiệm: Việc ứng dụng linh hoạt giữa mô hình quan hệ chặt chẽ (SQLite/SQLAlchemy) và định dạng dữ liệu bán cấu trúc (JSON) trong lưu trữ báo cáo Heuristic cho thấy sự tối ưu hóa cao độ về mặt lưu trữ truy vấn. Song song đó, triết lý thiết kế UI/UX hiện đại (Modern Minimalism) với việc thị giác hóa rủi ro qua mã màu đã đập tan rào cản kỹ thuật, giúp bất kỳ người dùng phổ thông nào cũng có thể dễ dàng tương tác và nắm bắt các phán quyết của AI.

Thứ ba, về mặt Thiết kế Thuật toán Lỗi: Lưu đồ thuật toán tuần tự mô tả pipeline **PhoBERT Text-only + MLP**: embedding 768 chiều → StandardScaler → MLPClassifier (scikit-learn) → verdict 3 mức → ExplanationEngine rule-based.

Tóm lại, những bản thiết kế logic mạch lạc, từ cấp độ Cơ sở dữ liệu (ERD) cho đến sơ đồ Thuật toán lỗi được vạch ra trong Chương 3 chính là kim chỉ nam vững chắc. Khung kiến trúc kỹ thuật này đã sẵn sàng để được chuyển hóa thành những dòng mã nguồn thực tế. Toàn bộ quá trình lập trình cài đặt (Implementation) cũng như chạy các thực nghiệm đánh giá hiệu năng mô hình (Evaluation) sẽ được trình bày cặn kẽ trong **Chương 4: Hiện thực và Kết quả**.

CHƯƠNG 4: HIỆN THỰC VÀ KẾT QUẢ

4.1. Môi trường phát triển và Thực nghiệm

Để đảm bảo khả năng hiện thực hóa các bản thiết kế kiến trúc phức tạp ở Chương 3, đặc biệt là việc huấn luyện và chạy suy luận (Inference) cho các mô hình Học Sâu (Deep Learning) đòi hỏi khối lượng tính toán lớn, đồ án đã được triển khai và kiểm thử trên một môi trường máy tính cục bộ (Local Environment) có cấu hình kỹ thuật tiêu chuẩn. Việc thiết lập môi trường được chia thành hai mảng chuyên biệt: Phần cứng và Phần mềm.

4.1.1. Môi trường Phần cứng (Hardware Environment)

Quá trình huấn luyện mạng nơ-ron đa lớp và kiến trúc Transformer [6, 7] (PhoBERT) đòi hỏi năng lực xử lý ma trận khổng lồ mà các bộ vi xử lý thông thường không thể đáp ứng được trong một thời gian hợp lý. Do đó, hệ thống phần cứng được thiết lập khắt khe với các thông số sau:

- **Vi xử lý trung tâm (CPU):** Tối thiểu Intel Core i5 hoặc AMD Ryzen 5 thế hệ mới, đáp ứng tốt quá trình cào dữ liệu (Web Scraping) đa luồng bằng BeautifulSoup và xử lý tiền dữ liệu văn bản bằng Regex.
- **Bộ nhớ trong (RAM):** Tối thiểu **16GB**. Dung lượng này là ranh giới bắt buộc để hệ thống có thể tải đồng thời tập dữ liệu huấn luyện (Dataset) vào bộ nhớ chính, đồng thời duy trì các máy chủ ảo cục bộ (cả FastAPI và Next.js) hoạt động song song mà không bị hiện tượng tràn RAM (Out-of-memory crash).
- **Bộ xử lý Đồ họa (GPU):** Khuyến nghị card **NVIDIA** hỗ trợ **CUDA** để tăng tốc suy luận PhoBERT (embedding). MLP huấn luyện bằng scikit-learn trên CPU; GPU chủ yếu phục vụ trích embedding hàng loạt khi train offline.

4.1.2. Môi trường Hệ điều hành và Phần mềm

Để tạo ra một nền tảng vận hành mượt mà, hạn chế tối đa các xung đột thư viện (Dependency conflicts) thường gặp trong quá trình biên dịch mô hình AI, đồ án đã lựa chọn và chốt cấu hình các công nghệ phần mềm cốt lõi như sau:

- **Hệ điều hành cốt lõi: Linux (Ubuntu)** — tương thích mã nguồn mở, khởi chạy script điều phối (`setup.sh` , `run.sh`) ổn định.
- **Hệ sinh thái Backend & AI:**
 - **Ngôn ngữ:** Sử dụng phiên bản `Python 3.10+` (toàn bộ mã nguồn chạy bên trong một môi trường ảo Virtual Environment riêng biệt để cách ly các thư viện).
 - **Framework máy chủ:** `FastAPI` kết hợp cùng máy chủ ASGI `Uvicorn` để kích hoạt khả năng xử lý bất đồng bộ (Asynchronous) ở tốc độ cao nhất.
 - **Thư viện AI Lõi:** `PyTorch` + `Transformers` (PhoBERT embedding), `Scikit-learn` (`MLPClassifier` , `StandardScaler` , metrics `F1/Precision`), `joblib` (luu artifact).
- **Hệ sinh thái Frontend:** Được xây dựng trên nền tảng môi trường cục bộ `Node.js` . Sử dụng ngôn ngữ `TypeScript` nhằm ép buộc tính an toàn kiểu dữ liệu (Type-safety). Khung thiết kế chính là `Next.js 14` kết hợp với hệ sinh thái `React` và bộ thư viện tiện ích `Tailwind CSS` .
- **Hệ quản trị Cơ sở dữ liệu:** Để phục vụ mục đích thực nghiệm và phát triển (Development Phase) linh hoạt, đồ án triển khai `CSDL SQLite` thay vì cài đặt các hệ thống máy chủ công kênh như `MySQL` hay `PostgreSQL`. Mọi giao tiếp nhúng với `CSDL` được thực hiện gián tiếp thông qua thư viện ORM `SQLAlchemy` .

4.2. Quá trình hiện thực

4.2.1. Cài đặt các module lõi của hệ thống (Core Modules Implementation)

Mã nguồn của hệ thống được tổ chức khoa học theo nguyên tắc **Separation of Concerns (Tách biệt mối quan tâm)**. Thay vì gộp chung toàn bộ logic vào một khối lệnh nguyên khối khó kiểm soát, đồ án đã phân rã hệ thống thành các module Python độc lập. Lối kiến trúc này giúp mã nguồn dễ bảo trì, dễ kiểm thử (Unit Test) và dễ dàng nâng cấp thuật toán trong tương lai. Các module lõi được cài đặt như sau:

- **Module Thu thập và Tiền xử lý (data_crawler.py , text_utils.py)**: Khi người dùng nhập URL, data_crawler.py dùng BeautifulSoup + requests trích title/content từ HTML. Văn bản thu được (hoặc nhập trực tiếp) đi qua preprocess_text trong text_utils.py. Helper TextCleaner (HTML/teencode) chỉ dùng trong test, không nằm luồng chính.
- **Module Suy luận Phân loại (phobert_inference.py)**: Tải PhoBERT-base qua Transformers (PyTorch), trích vector [CLS] 768 chiều, chuẩn hóa bằng StandardScaler và suy luận qua MLPClassifier đã lưu (.joblib). predict_proba trả xác suất tin giả; verdict.py quy đổi 3 mức cảnh báo.
- **Module Giải thích (explanation_engine.py)**: Chạy sau phân loại. Quét văn bản gốc (tỷ lệ IN HOA, dấu câu, văn phong giật gân), áp luật rule-based và sinh JSON giải thích tiếng Việt — không tham gia vector đầu vào MLP.

4.2.2. Dữ liệu nghiên cứu và Quy trình Tiền xử lý

Giới thiệu bộ dữ liệu (Dataset Overview)

Để quá trình huấn luyện mạng MLP mang lại độ chính xác cao nhất trong thực tế, đồ án sử dụng bộ dữ liệu tin giả y tế/sức khỏe tiếng Việt trong tệp full_dataset.csv — **10.617 bản ghi** gốc, **10.609 mẫu** sau lọc. Dữ liệu được thu thập từ các nguồn báo chí và tập COVID-19 tiếng Việt (VnExpress, daikynghuyen, covid19_dataset, ...), tập trung chủ đề y tế và sức khỏe tại Việt Nam.

Biện luận lý do lựa chọn bộ dữ liệu quy mô lớn

Việc quyết định sử dụng bộ dữ liệu đa lĩnh vực này xuất phát từ ba luận điểm mang tính chiến lược:

- **Tính khái quát hóa cao (Generalization)**: Khác với các hệ thống chỉ học trên một miền dữ liệu hẹp, việc học trên **10.609 mẫu** tin y tế/sức khỏe tiếng Việt giúp mô hình nắm bắt ngữ cảnh chuyên ngành và có thể áp dụng kiểm chứng thông tin sức khỏe trên không gian mạng.
- **Độ phong phú của thuật ngữ**: Ngữ liệu lớn chứa rất nhiều từ vựng đan xen với cấu trúc câu mang tính mỗi chài của giới bán hàng giả hoặc câu view giật gân. Đây là một môi trường thử nghiệm phù hợp để mô hình PhoBERT [1] học cách phân biệt ngữ cảnh tiếng Việt phức tạp.
- **Sự phù hợp với phương pháp Heuristics**: Các tin giả trên không gian mạng thường xuyên lặp lại các hành vi thao túng tâm lý (như viết hoa toàn bộ tựa đề để lôi kéo, dùng hàng loạt dấu chấm than để kích động). Do đó, bộ dữ liệu khổng lồ này chính là "đất diễn" hiệu quả để Động cơ Giải thích Heuristics của đồ án phát huy tối đa sức mạnh.

Quy trình Tiền xử lý Dữ liệu Huấn luyện (Data Preprocessing Workflow)

Trước khi huấn luyện MLP, toàn bộ tập dữ liệu thô trải qua quy trình làm sạch và chia tập:

- **Làm sạch (dataset_cleaner.py)**: Tiến hành lọc và loại bỏ chính xác 8 bản ghi không đạt yêu cầu. Các lỗi dẫn đến việc loại bỏ bao gồm: văn bản bị rỗng, dữ liệu bị trùng lặp hoàn toàn (duplicates), thiếu nhãn phân loại, hoặc lỗi định dạng chuỗi (encoding) trong quá trình thu thập. Kết quả thu về **10.609 mẫu** hoàn toàn hợp lệ.
- **Tách tập huấn luyện (Train/Test Split)**: Chia **70% huấn luyện — 30% kiểm thử** (stratified, random_state=42), tương đương ~**3.183 mẫu** hold-out.

- **Trích embedding & huấn luyện MLP:** Mỗi mẫu qua `preprocess_text` → PhoBERT [CLS] 768-d → `StandardScaler.fit` trên train → `MLPClassifier.fit` (128, 64). Artifact lưu `.joblib`. Không dùng PyTorch DataLoader hay resampling bắt buộc — tập đã cân bằng tương đối.

4.2.3. Tổ chức cấu trúc mã nguồn (Project Directory Structure)

Để đảm bảo tính mở rộng, dễ bảo trì và làm việc nhóm hiệu quả, toàn bộ mã nguồn của dự án được tổ chức chặt chẽ theo mô hình phân tách (Decoupled). Dưới đây là sơ đồ cây thư mục (Directory Tree) minh họa cấu trúc các thành phần cốt lõi của hệ thống ShieldAI:

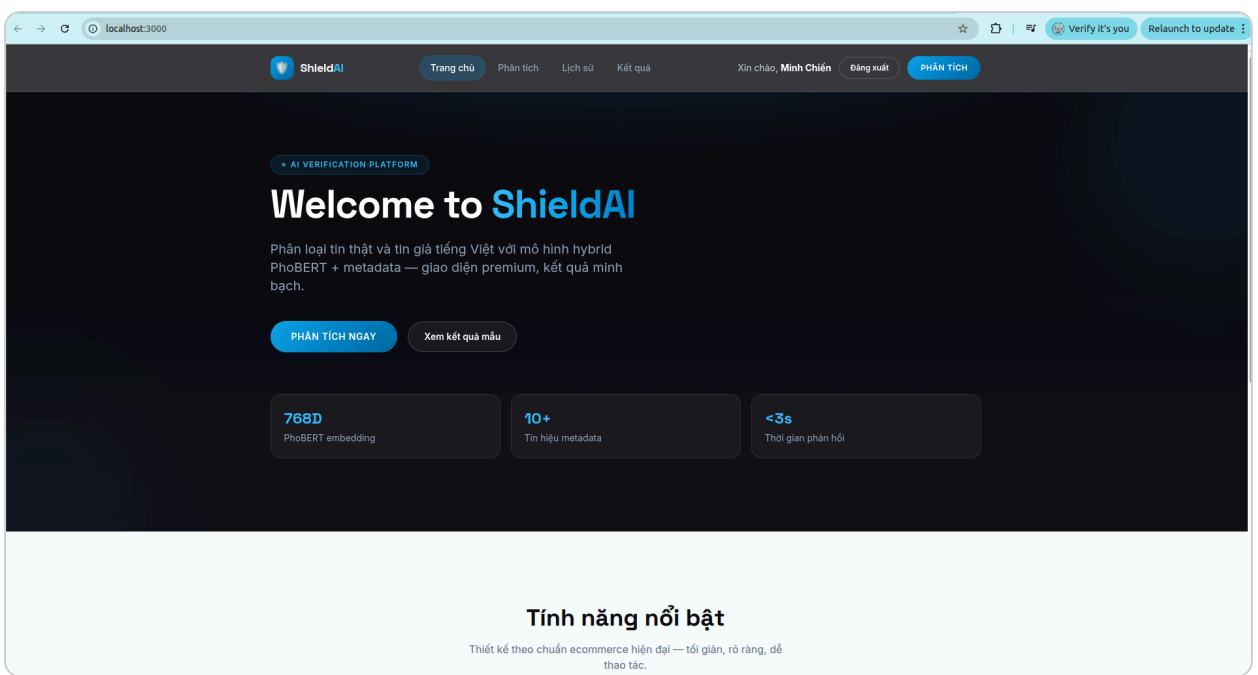
```
ShieldAI_Project/
├── backend/                                # Khôi Máy chủ API (Python & FastAPI)
│   ├── api/                               # Khai báo các API Endpoints (Auth, Analyze, History)
│   ├── auth/                             # Xử lý bảo mật, mã hóa JWT, phân quyền
│   ├── database/                         # Kết nối CSDL SQLite và quản lý Models (SQLAlchemy)
│   ├── experiments/                     # Lưu trữ kết quả thực nghiệm, biểu đồ đánh giá mô hình
│   ├── models/                          # Artifact MLP + StandardScaler (.joblib); cache PhoBERT
│   ├── tests/                           # Bộ kịch bản kiểm thử tự động (Pytest) [14]
│   └── training/                         # Chứa Jupyter Notebook đề huấn luyện mô hình
│   (train_phobert_model.ipynb)
│   ├── data_crawler.py                  # Module cào dữ liệu thô từ Internet
│   ├── dataset_cleaner.py               # Module tiền xử lý và làm sạch dữ liệu thô
│   └── explanation_engine.py            # Động cơ Giải thích: Bóc tách và định lượng nguyên nhân lừa
đảo
│   ├── text_utils.py                    # preprocess_text, PyVi; helper TextCleaner (test)
│   ├── phobert_inference.py             # PhoBERT embedding + joblib MLP + verdict
│   └── main.py                          # Điểm khởi chạy (Entry point) của máy chủ FastAPI
├── docs/                                # Tài liệu kỹ thuật, Hướng dẫn cài đặt và Nhật ký dự án
├── frontend/                             # Khôi Giao diện Người dùng (TypeScript & Next.js)
│   ├── app/                             # Kiến trúc App Router của Next.js (chứa các trang /login,
/analyze,...)
│   ├── components/                     # Các Component tái sử dụng (Navbar, Chart, HeroBanner)
│   ├── context/                         # Quản lý trạng thái toàn cục (React Context API cho Auth)
│   ├── lib/                             # Các hàm tiện ích (gọi API, xử lý hiệu ứng Motion)
│   └── tailwind.config.ts               # Cấu hình hệ thống thiết kế (Design System & Colors)
├── scripts/                              # setup.sh, run.sh (all | api | web | test)
├── setup.sh / run.sh                    # Wrapper gộp scripts/
├── README.md                            # Tài liệu tổng quan giới thiệu dự án
└── requirements.txt                      # Danh sách các thư viện Python phụ thuộc
```

Mô hình tổ chức này tuân thủ nguyên tắc *Separation of Concerns* (Tách biệt mối quan tâm), giúp đội ngũ phát triển dễ dàng khoanh vùng lỗi, thực hiện kiểm thử độc lập ở Backend mà không ảnh hưởng tới quá trình thiết kế UI/UX ở Frontend.

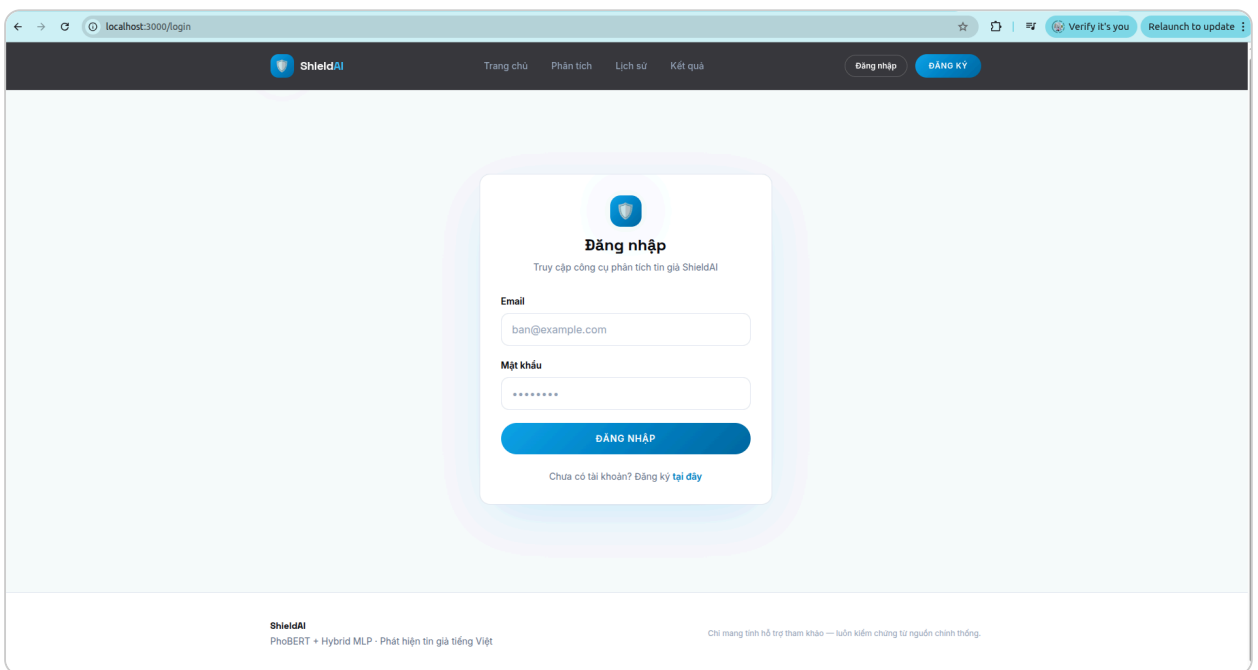
4.3. Kết quả đạt được

4.3.1. Kết quả giao diện ứng dụng (User Interface Implementation)

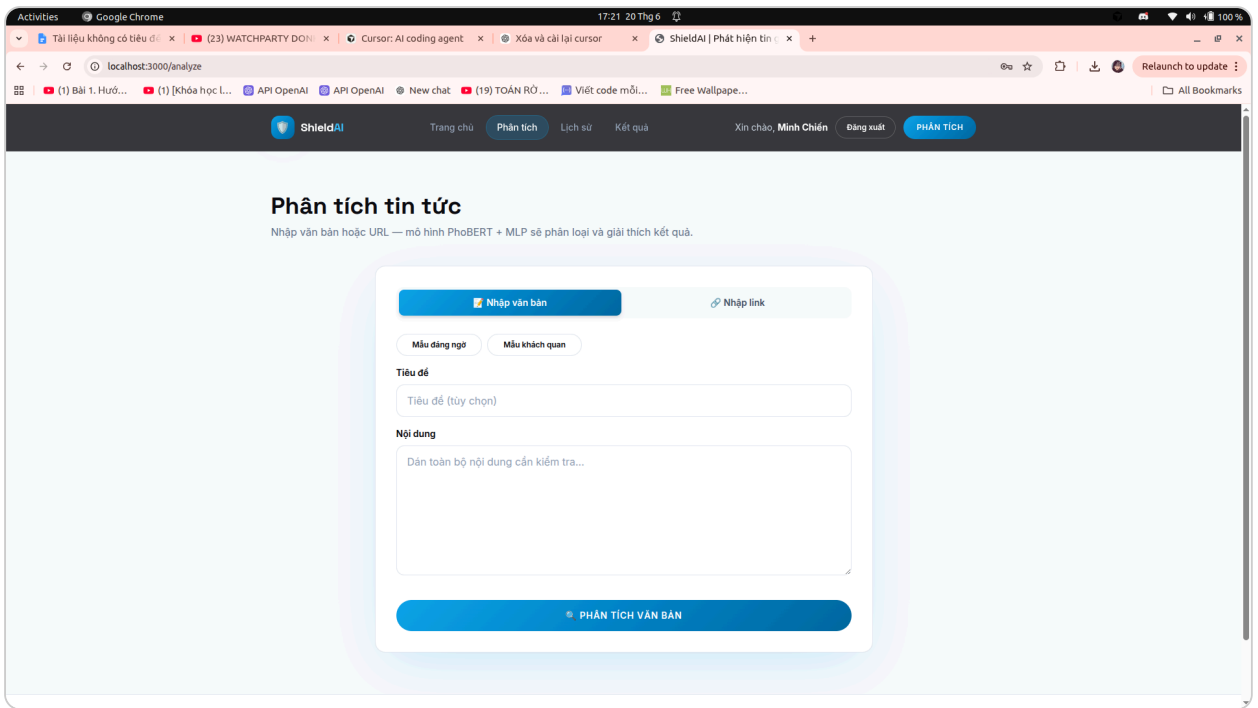
Hệ thống đã được lập trình hoàn thiện bằng Next.js và triển khai thành công trên môi trường cục bộ. Giao diện thực tế hoạt động trơn tru, đáp ứng chính xác các yêu cầu về thiết kế UI/UX (Modern Minimalism) đã đề ra ở Chương 3. Dưới đây là các minh chứng hình ảnh chụp lại từ hệ thống thực tế đang vận hành:



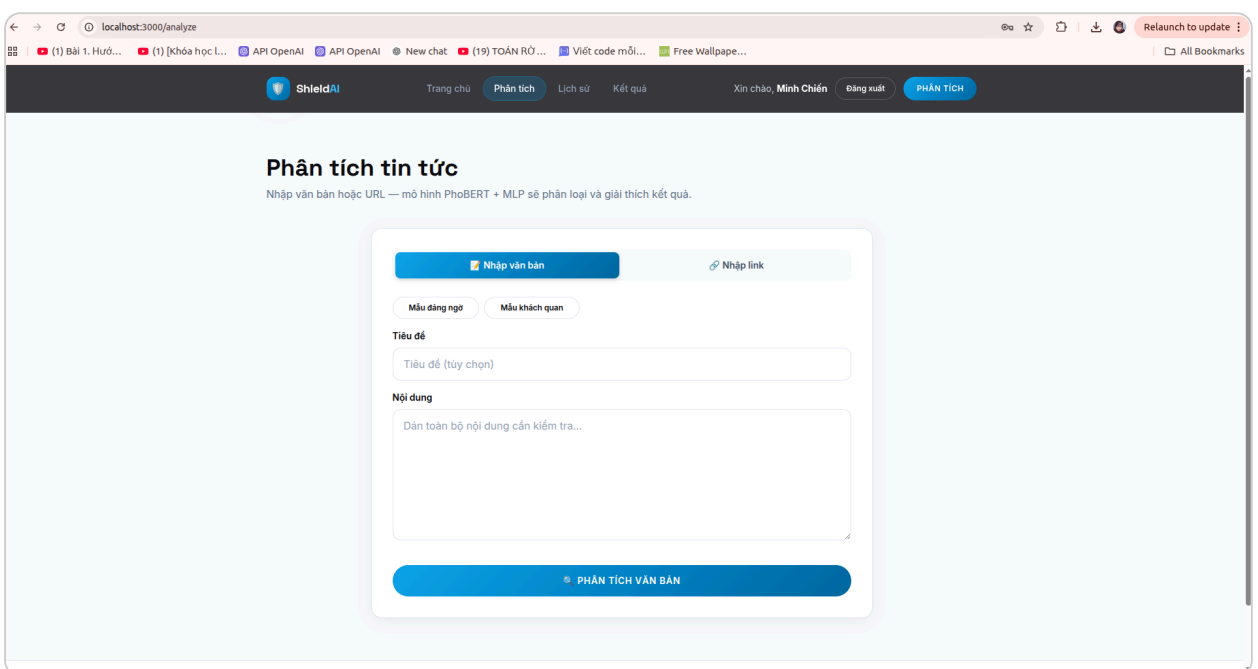
Hình 4.1. Giao diện màn hình Cổng thông tin (Home) với thiết kế tối giản, tông màu tối chuyên nghiệp.



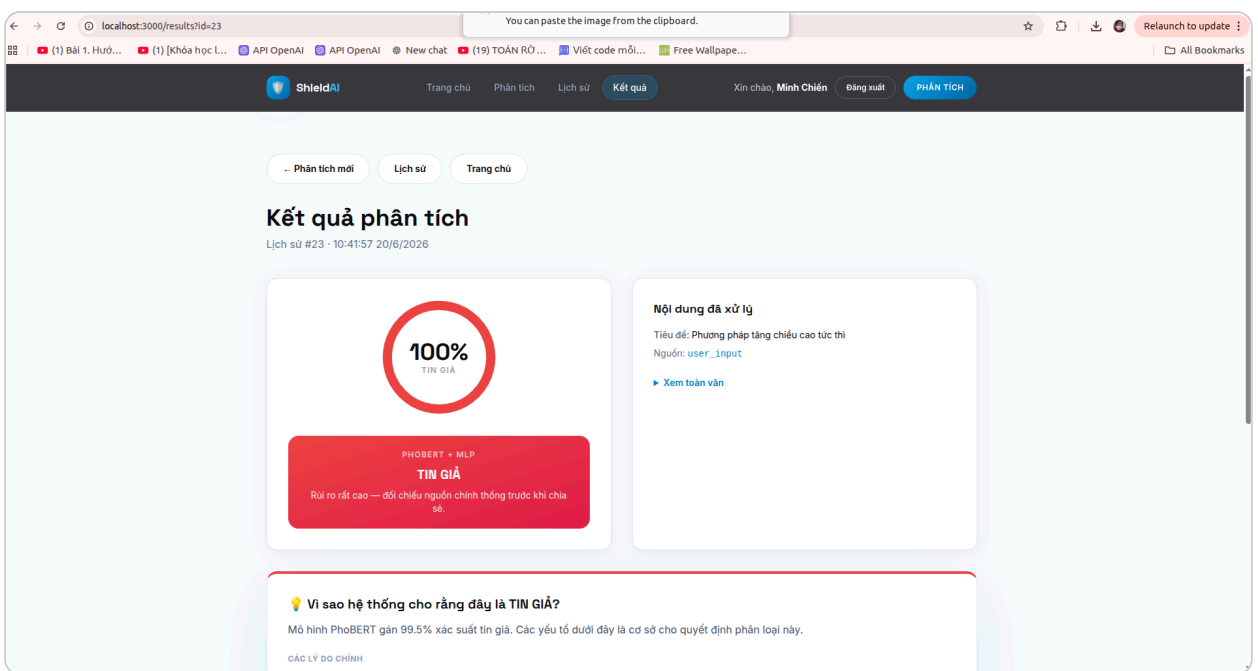
Hình 4.2. Giao diện Đăng nhập (Authentication) bảo mật người dùng.



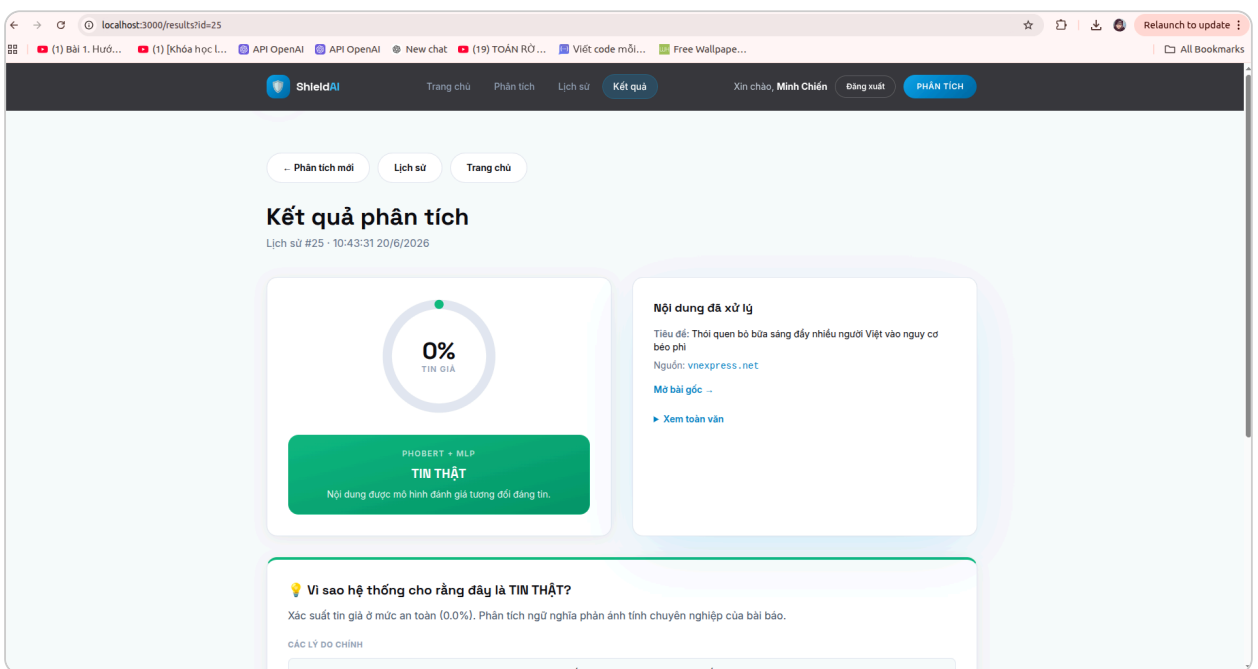
Hình 4.3. Giao diện trung tâm Phân tích — Hỗ trợ nhập văn bản trực tiếp để kiểm chứng.



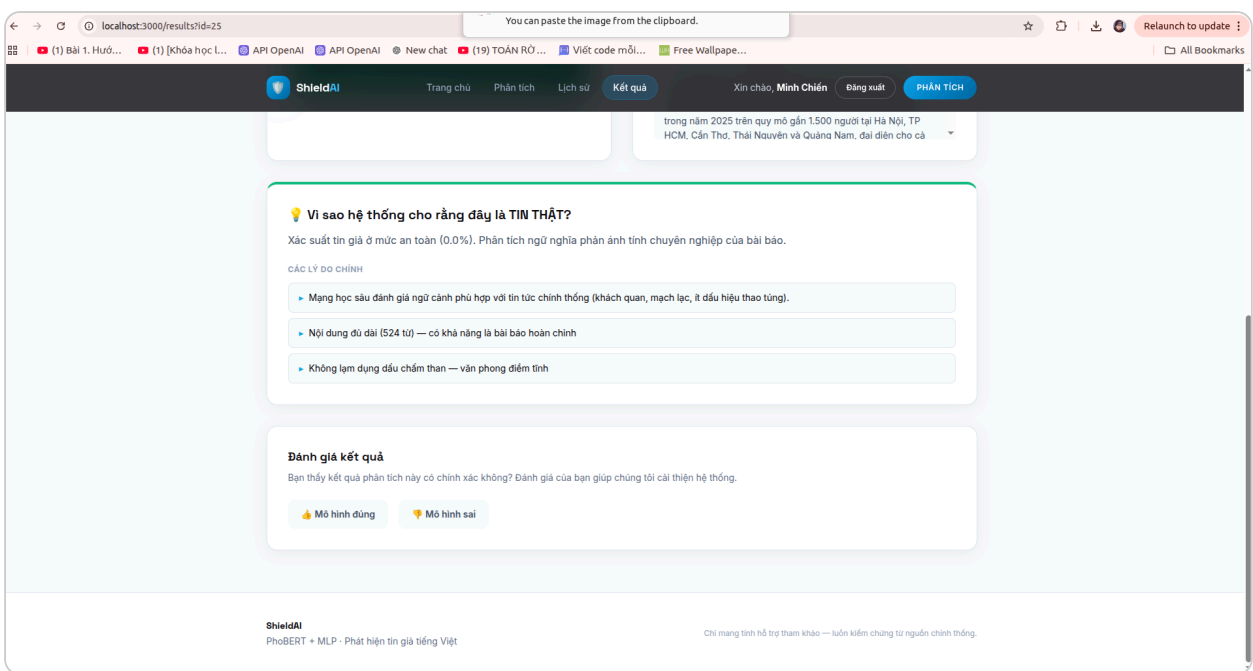
Hình 4.4. Giao diện trung tâm Phân tích — Hỗ trợ dán liên kết (URL) của bài báo.



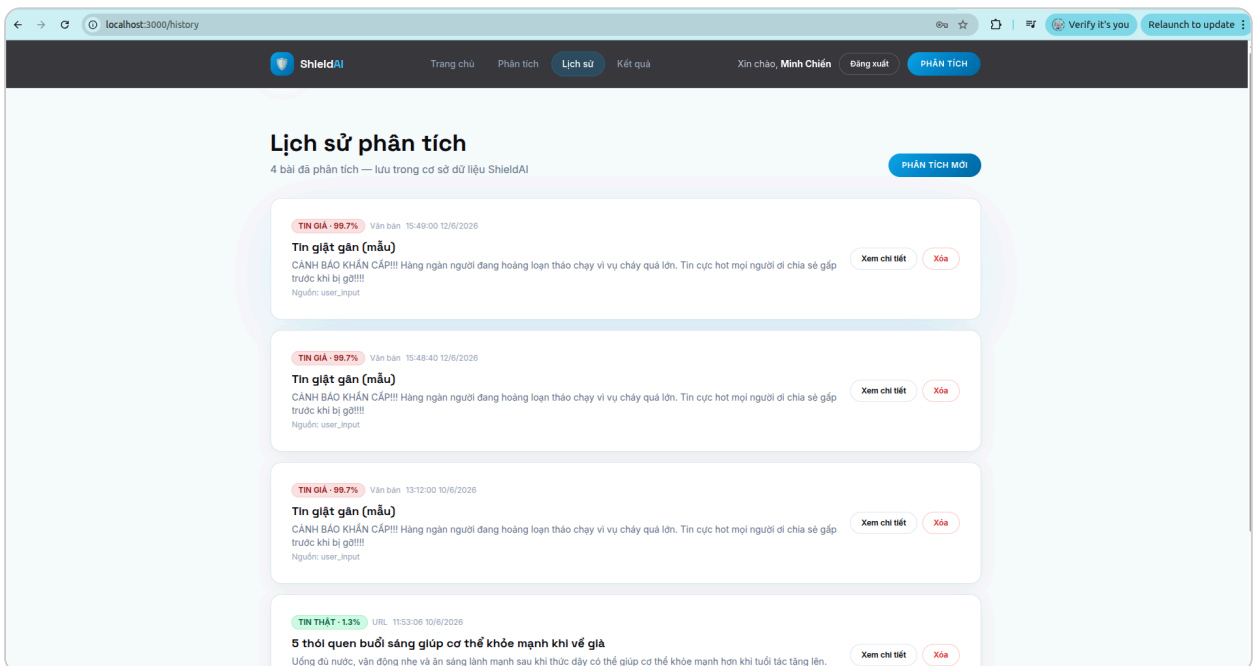
Hình 4.5. Báo cáo phân tích cảnh báo nguy cơ Tin Giả thông qua nguyên lý Thị giác hóa rủi ro (Màu Đỏ - Mức độ 100%).



Hình 4.6. Báo cáo phân tích xác thực Tin Thật an toàn (Màu Xanh lá cây - Mức độ 0%).



Hình 4.7. Trình diễn khả năng của Động cơ Giải thích: Hệ thống bóc tách rõ lý do bài báo được đánh giá là chuyên nghiệp và an toàn.



Hình 4.8. Màn hình Quản lý Lịch sử cá nhân với thiết kế dạng danh sách thẻ động (Dynamic Cards).

4.3.2. Đánh giá kết quả bằng số liệu thực nghiệm

Để minh chứng tính đúng đắn của phương pháp tiếp cận, đồ án không chỉ dựa vào đánh giá cảm quan trên giao diện mà còn tiến hành định lượng (Quantitative Evaluation) hệ thống bằng các độ đo học máy chuẩn mực. Thử nghiệm được tiến hành trên tập kiểm thử hold-out **30%** (~3.183 mẫu) chưa từng được mô hình nhìn thấy trong quá trình huấn luyện.

a) Các độ đo đánh giá (Evaluation Metrics)

Hệ thống sử dụng ma trận nhầm lẫn (Confusion Matrix) để tính toán 4 chỉ số cốt lõi:

- **Accuracy (Độ chính xác tổng thể):** Tỷ lệ phần trăm các bài báo (cả thật và giả) được hệ thống phân loại đúng. Tuy nhiên, trong bài toán tin giả, chỉ số này không phản ánh toàn diện bức tranh nếu dữ liệu bị mất cân bằng.
- **Precision (Độ chuẩn xác):** Trong số những bài báo bị hệ thống gắn cờ "Tin Giả", có bao nhiêu bài thực sự là giả. Độ đo này đóng vai trò quan trọng trong việc hạn chế hiện tượng *Dương tính giả (False Positive)*, tức là đánh oan một bài báo chính thống.
- **Recall (Độ bao phủ):** Trong tổng số các bài Tin Giả đang tồn tại trong thực tế, hệ thống "tóm" được bao nhiêu phần trăm. Độ đo này giúp đánh giá khả năng không bỏ lọt tội phạm (*Âm tính giả - False Negative*).
- **F1-Score:** Trung bình điều hòa (Harmonic Mean) giữa Precision và Recall. Đây là "thước đo vàng" (Gold Standard) để đánh giá năng lực thực sự của một hệ thống phát hiện tin giả.

b) Kết quả so sánh trên tập kiểm thử

Để đảm bảo tính khách quan trong việc đánh giá hiệu năng, toàn bộ **10.609 mẫu** sau lọc đã được chia tách theo tỷ lệ **70% (Huấn luyện) — 30% (Kiểm thử)**, tương đương khoảng **~3.183 mẫu** tập kiểm thử hold-out. Quá trình chia tách sử dụng phương pháp phân tầng (Stratified Split) với `random_state=42` để đảm bảo phân phối nhãn (Thật/Giả) đồng đều, ngăn chặn hiện tượng rò rỉ dữ liệu (Data Leakage). Quá trình huấn luyện còn được củng cố bởi phương pháp kiểm định chéo 5-Fold Cross Validation.

Đồ án đã tiến hành đánh giá chi tiết hiệu năng của mô hình **PhoBERT Text-only + MLP (Hệ thống ShieldAI)** trên tập kiểm thử độc lập (chiếm 30% dữ liệu). Kết quả được trình bày chi tiết trong Bảng 4.1:

Bảng 4.1. Hiệu năng của mô hình PhoBERT Text-only + MLP

Mô hình (Model)	Accuracy	F1-Score	ROC-AUC	5-Fold CV (F1)
PhoBERT Text-only + MLP	94.13%	93.71%	98.50%	93.17% ± 1.33%

Từ Bảng 4.1, có thể thấy kiến trúc của hệ thống đạt F1-Score 93,71% và ROC-AUC 98,50%. Việc tách biệt các vector Heuristics giúp mô hình học sâu tập trung vào khả năng phân tích ngữ nghĩa, đạt F1-Score 93,71% trên tập kiểm thử hold-out (~3.183 mẫu, 30% của 10.609 mẫu).

c) Phân tích sự ổn định qua Kiểm định chéo (Cross-validation)

Để đánh giá tổng quan hơn, đồ án áp dụng phương pháp **Kiểm định chéo 5 phần (5-Fold Cross Validation)**. Kết quả ghi nhận **F1-Score đạt 93.17% ± 1.33%** (trong đó, giá trị $\pm 1.33\%$ đại diện cho độ lệch chuẩn - Standard Deviation giữa các vòng lặp fold). Sự dao động biên độ hẹp này là một minh chứng quan trọng, cho thấy hiệu năng của hệ thống cực kỳ ổn định và ít phụ thuộc vào cách chia tách dữ liệu ngẫu nhiên. Về phía chỉ số **ROC-AUC đạt 98.50%**, đây là một con số rất ấn tượng, phản ánh khả năng phân tách hai lớp Thật/Giả của mạng Nơ-ron đa tầng MLP là vô cùng xuất sắc. Việc có sự chênh lệch khá rõ rệt giữa ROC-AUC (98,50%) và F1-Score (93,71%) là một hiện tượng hoàn toàn hợp lý về mặt khoa học dữ liệu: nguyên nhân cốt lõi là do ROC-AUC đánh giá khả năng phân tách tổng quát của mô hình trên toàn bộ các ngưỡng quyết định (thresholds) có thể có, trong khi đó F1-score lại là một độ đo phụ thuộc nghiêm ngặt vào một ngưỡng phân loại cố định (mặc định là 0.5). Do đó, hai chỉ số này mang ý nghĩa bổ trợ cho nhau chứ không nhất thiết phải bằng nhau.

4.4. Kiểm thử hệ thống (System Testing)

Để đảm bảo độ tin cậy và tính bền vững của phần mềm trước khi nghiệm thu, đồ án đã triển khai một quy trình kiểm thử cơ bản, tập trung vào các kịch bản trọng yếu thông qua kiểm thử tự động (Automated Testing) và kiểm thử thủ công (Black-box Testing).

4.4.1. Kiểm thử tự động bằng Pytest (Automated Testing)

Hệ thống Backend được tích hợp bộ kiểm thử `pytest` trong thư mục `backend/tests/`. Tổng cộng **42 kịch bản (test cases)** chạy tự động, gồm kiểm thử đơn vị (unit test) và kiểm thử tích hợp API (integration test). Các test API dùng SQLite in-memory và mock `PhoBERTInferenceSystem`; lỗi nghiệp vụ (auth, input trống) được assert qua `status=error` trong body JSON với HTTP 200.

Cách chạy: `./run.sh test` hoặc `cd backend && ./run_tests.sh`. Kết quả hiện tại: **42/42 PASSED** (~4 giây).

Bảng 4.3. Bộ kiểm thử tự động Pytest (backend/tests/)

STT	File test	Số TC	Phạm vi kiểm thử	Kỹ thuật / Kỳ vọng
1	test_verdict.py	5	Phân loại 3 mức từ xác suất	Ngưỡng $\geq 75\%$ tin giả, 35–74% đáng ngờ, $< 35\%$ tin thật; kiểm tra biên 75/74.9/35/34.9
2	test_text_utils.py	5	Pipeline preprocess_text	Xóa URL, lowercase, PyVi; thống nhất train/inference/dataset_cleaner
3	test_text_cleaner.py	3	Helper TextCleaner (không luồng inference chính)	HTML/teencode; TC-09 đối chiếu với preprocess_text
4	test_explanation_engine.py	6	Động cơ giải thích XAI	Ngưỡng model note khớp verdict; phát hiện văn phong giật gân; build_explanation JSON
5	test_dataset_cleaner.py	3	Lọc CSV trước train	Bỏ thiếu content, < 5 từ, trùng lặp; tạo cột content_segmented
6	test_data_crawler.py	3	Thu thập văn bản / URL	Nhập text thủ công; mock HTTP crawl HTML $\rightarrow$ parse h1/p
7	test_phobert_inference.py	4	Pipeline suy luận 3 mô-đun	Mock PhoBERT/MLP: thiếu model, từ chối < 5 từ, phân loại fake, luồng infer() đầy đủ
8	test_history_service.py	1	Lưu trữ SQLite	save_analysis + list_user_history trả đúng verdict/prob
9	test_auth_api.py	4	API xác thực JWT	Đăng ký/đăng nhập//me; email trùng; sai mật khẩu
10	test_api.py	8	API phân tích & lịch sử	Health, analyze (text/url), lịch sử CRUD, cô lập dữ liệu giữa 2 user
Tổng cộng		42	Unit: 30 API integration: 12 Fixture: conftest.py (DB test + mock inference)	

Bảng 4.3 tổng hợp theo file test. Dưới đây, **42 kịch bản** được phân thành **8 nhóm chức năng** và liệt kê chi tiết tại Bảng 4.4 (ánh xạ FR từ Bảng 3.1):

- **Nhóm 1 — Verdict 3 mức** (TC-01 $\rightarrow$ TC-05, test_verdict.py): kiểm tra ngưỡng 75% / 35% và giá trị biên.
- **Nhóm 2 — Tiền xử lý văn bản** (TC-06 $\rightarrow$ TC-13): preprocess_text (TC-06–08, TC-10); helper TextCleaner (TC-09, TC-11–13, không nằm luồng inference chính).
- **Nhóm 3 — Giải thích XAI** (TC-14 $\rightarrow$ TC-19, test_explanation_engine.py): ngưỡng giải thích khớp verdict.py, phát hiện giật gân.
- **Nhóm 4 — Dữ liệu & Crawl** (TC-20 $\rightarrow$ TC-25): lọc CSV, nhập text, crawl HTML mock.
- **Nhóm 5 — Pipeline suy luận** (TC-26 $\rightarrow$ TC-29, test_phobert_inference.py): 3 mô-đun + luồng infer() (mock PhoBERT).
- **Nhóm 6 — Lịch sử SQLite** (TC-30, test_history_service.py): lưu và đọc kết quả phân tích.
- **Nhóm 7 — API xác thực** (TC-31 $\rightarrow$ TC-34, test_auth_api.py): đăng ký, đăng nhập, JWT.
- **Nhóm 8 — API phân tích & lịch sử** (TC-35 $\rightarrow$ TC-42, test_api.py): health, analyze, CRUD history, cô lập user.

4.4.1.1. Phân nhóm và chi tiết 42 test cases

Bảng 4.4. Chi tiết từng kịch bản kiểm thử tự động (42 test cases)

Mã	Hàm test	Loại	FR	Đầu vào / Kịch bản	Kỳ vọng (assert)
TC-01	test_verdict_thresholds[80]	Unit	FR-05	Xác suất tin giả = 80%	verdict=fake (FAKE)
TC-02	test_verdict_thresholds[60]	Unit	FR-05	Xác suất tin giả = 60%	verdict=suspicious ĐANG NGỜ
TC-03	test_verdict_thresholds[45]	Unit	FR-05	Xác suất tin giả = 45%	verdict=suspicious ĐANG NGỜ
TC-04	test_verdict_thresholds[30]	Unit	FR-05	Xác suất tin giả = 30%	verdict=real THẬT (REAL)
TC-05	test_verdict_boundary_values	Unit	FR-05	Biên ngưỡng: 75, 74.9, 35, 34.9	75→fake; 74.9→suspicious; 35→suspicious; 34.9→real
TC-06	test_preprocess_text_basic	Unit	FR-03	Văn bản chứa URL https://example.com	Không còn URL http/example.com PyVi ()
TC-07	test_segment_for_training_matches_preprocess_text	Unit	FR-03	Câu y tế mẫu	segment_for_training_matches_preprocess_text
TC-08	test_dataset_cleaner_uses_same_pipeline	Unit	FR-03	Text giặt gân + URL	segment_text_matches_preprocess_text
TC-09	test_text_cleaner_pipeline_matches_preprocess_text	Unit	—	Helper TextCleaner vs preprocess_text	pipeline_clean_matches_preprocess_text
TC-10	test_preprocess_text_empty	Unit	FR-03	Chuỗi rỗng "" và None	Trả về "", None
TC-11	test_remove_html_and_urls	Unit	—	Helper: thẻ <a href> + URL	Xóa HTML và URL
TC-12	test_normalize_teencode	Unit	—	Helper: teencode wa, j, ng	→ quá, gì, ...
TC-13	test_pipeline_clean	Unit	—	Helper TextCleaner.pipeline_clean	Lowercase PyVi
TC-14	test_phobert_summary_aligns_with_verdict_thresholds	Unit	FR-06	Xác suất 80% / 50% / 20%	Tóm tắt chi tiết / chỉ / chỉ
TC-15	test_phobert_summary_boundary_at_fake_threshold	Unit	FR-06	Biên ngưỡng fake: 75% và 74.9%	75%→tin giả; 74.9%→tin giả
TC-16	test_phobert_summary_boundary_at_suspicious_threshold	Unit	FR-06	Biên ngưỡng suspicious: 35% và 34.9%	35%→dị thường; 34.9%→dị thường
TC-17	test_scan_text_signals_detects_sensational_language	Unit	FR-06	Tiêu đề + nội dung giật gân (CẢNH BÁO KHẨN CẤP)	Signal có
TC-18	test_build_explanation_matches_verdict_for_fake	Unit	FR-06	fake_prob=85, nội dung giật gân	verdict=fake chứa TIN GIẢ
TC-19	test_build_explanation_real_news_tone	Unit	FR-06	fake_prob=10, văn phong báo chí khách quan	verdict=real chứa TIN THẬT
TC-20	test_filter_dataset_removes_short_missing_and_duplicates	Unit	—	DataFrame 5 dòng: ngắn, thiếu, trùng	Còn 2 dòng content unique
TC-21	test_fill_missing_text_columns	Unit	—	DataFrame có content=None	title không None
TC-22	test_add_segmented_column_creates_pyvi_tokens	Unit	FR-03	DataFrame sau filter_dataset	Có cột content_tokens không rỗng

TC-23	test_get_article_from_text_success	Unit	FR-01	Nhập text + tiêu đề thủ công	status=suc source_dom
TC-24	test_get_article_from_text_empty	Unit	FR-01	Chuỗi chỉ khoảng trắng "	status=err
TC-25	test_crawl_news_article_pares_html	Unit	FR-02	Mock HTTP HTML có <h1> và <p>	Parse title source_dom
TC-26	test_infer_fails_when_model_not_loaded	Unit	—	model_loaded=False	status=err ý train m
TC-27	test_module_1_rejects_short_content	Unit	FR-03	Nội dung chỉ 3 từ	clean=None raw.status
TC-28	test_module_3_classification_fake_verdict	Unit	FR-05	Mock MLP trả xác suất 82%	verdict=fa nhân chứ
TC-29	test_infer_success_pipeline	Unit	FR-04,05	Mock module 1→2→3 + explanation	status=suc bước pipe
TC-30	test_save_and_list_analysis	Unit	FR-07	Lưu kết quả phân tích vào SQLite test	total=1, ve fake_prob=
TC-31	test_register_and_login_flow	API	—	POST register → login → GET /api/auth/me	Có JWT to ở cả 3 bư
TC-32	test_register_duplicate_email	API	—	Đăng ký 2 lần cùng email	Lần 2: sta message
TC-33	test_login_wrong_password	API	—	Đăng nhập sai mật khẩu	status=err
TC-34	test_me_requires_auth	API	—	GET /api/auth/me không có JWT	status=err
TC-35	test_health_check	API	FR-08	GET /api/health	status=ok, model_load
TC-36	test_analyze_unauthorized	API	FR-08	POST /api/analyze không JWT	status=err đăng nhậ
TC-37	test_analyze_empty_text_when_authenticated	API	FR-01	JWT hợp lệ, text=""	status=err chứa trồn
TC-38	test_analyze_text_success	API	FR-01,05,06,07	JWT + text giết gần đủ dài	status=suc fake_prob history_id
TC-39	test_analyze_url_mode	API	FR-02,05	mode=url + link VnExpress	status=suc infer(url=
TC-40	test_history_list_after_analyze	API	FR-07	Analyze xong → GET /api/history	total≥1, C
TC-41	test_history_detail_and_delete	API	FR-07	GET detail → DELETE → GET lại	Detail OK, GET sau b
TC-42	test_history_isolated_between_users	API	FR-07,08	User A tạo history; User B truy cập	User B nh (có lập d

Ghi chú cột FR: ánh xạ Bảng 3.1; — = kiểm thử helper (TextCleaner), auth nội bộ, hoặc pipeline dữ liệu. API lỗi nghiệp vụ: HTTP **200** + status=error + message (TC-32-34, TC-36-37, ...).

Đánh giá: 100% (42/42) test cases đạt PASSED. Kiểm thử API dùng mock inference nên không phụ thuộc GPU/RAM PhoBERT; DB test tách biệt file shieldai.db production.

4.4.2. Kiểm thử Hộp đen mức Ứng dụng (Manual Black-box Testing)

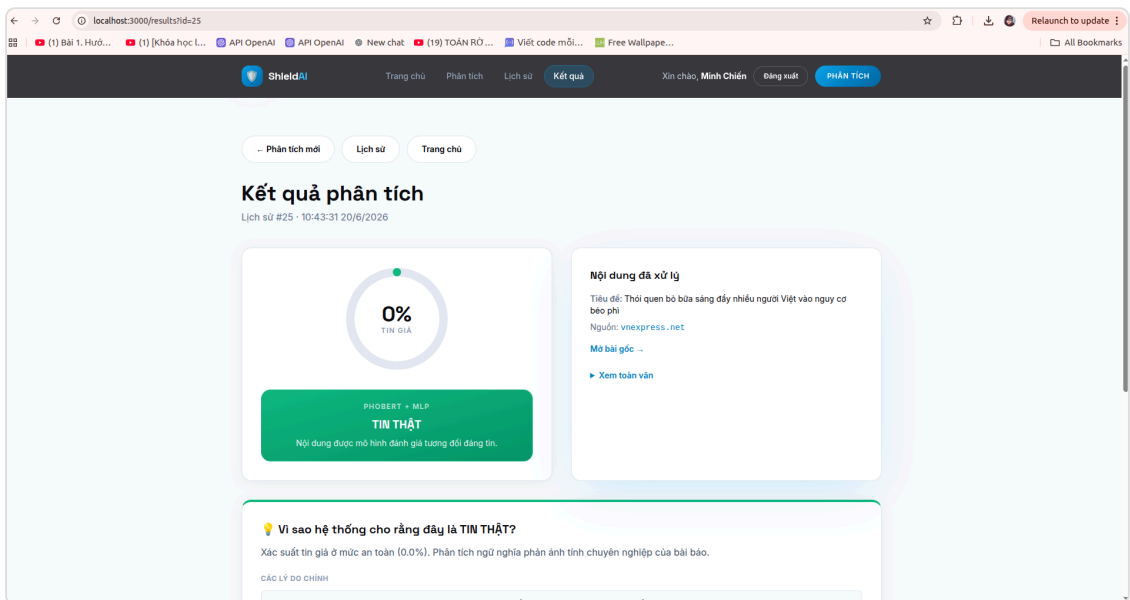
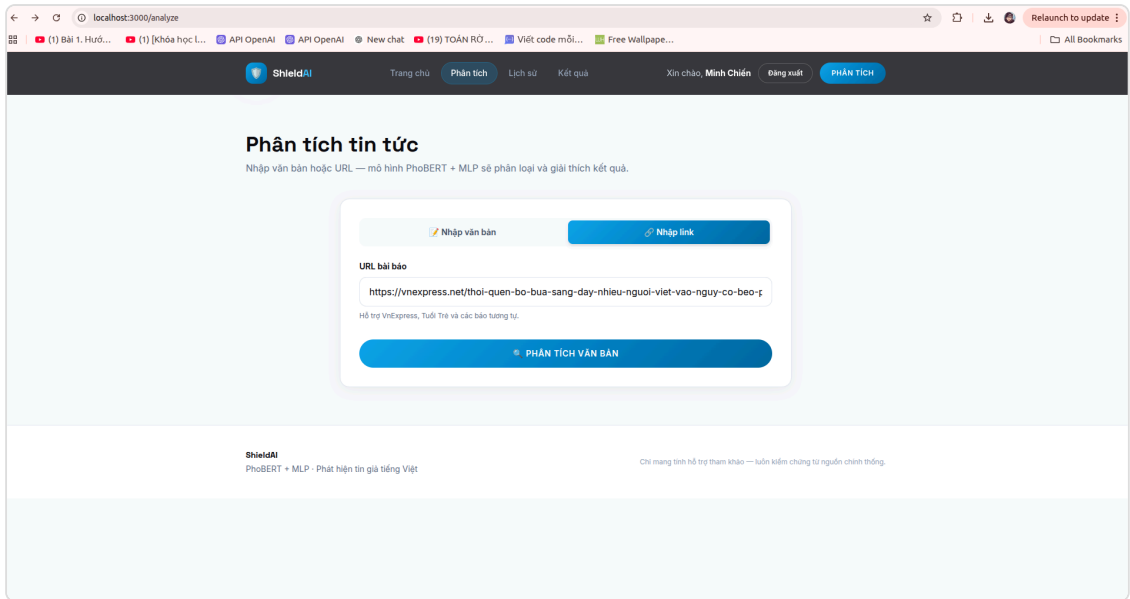
Ở tầng giao diện trực quan (Frontend), đồ án tiến hành kiểm thử các luồng thao tác thực tế nhằm đảm bảo trải nghiệm nghiệp vụ xuyên suốt, không bị đứt gãy. Bảng 4.2 dưới đây mô tả các kịch bản tiêu biểu:

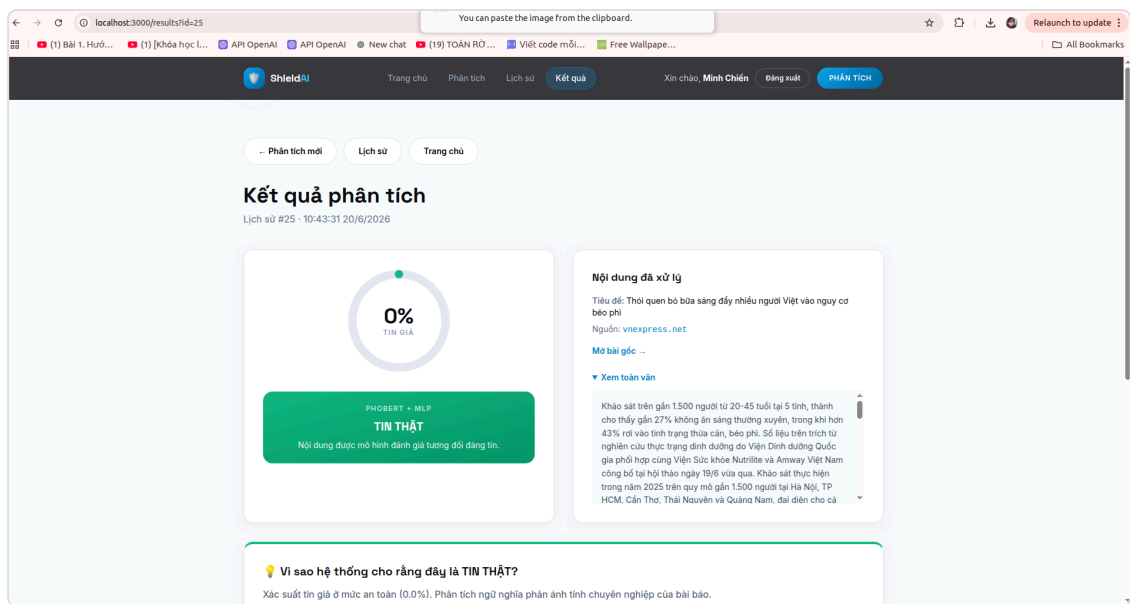
Bảng 4.2. Danh sách Kịch bản Kiểm thử Hộp đen

ID	Kịch bản (Test Scenario)	Kết quả mong đợi (Expected)	Kết quả thực tế	Status
TC_01	Nhập URL một bài báo chính thống hợp lệ từ báo điện tử VnExpress.	Hệ thống tải thành công tiêu đề, nội dung bài báo và hiển thị kết quả phân tích ổn định.	Cào đúng nội dung cốt lõi, tự động loại bỏ các đoạn quảng cáo (Ads).	PASS
TC_02	Nhập URL bị lỗi (404) hoặc nhập một trang web không phải trang tin tức (Ví dụ: Facebook cá nhân).	Hệ thống tự động báo lỗi "Không thể trích xuất nội dung" và chặn lệnh gửi phân tích cho AI.	Hiện thị thông báo Toast Notification rõ ràng trên giao diện người dùng.	PASS
TC_03	Gửi phân tích một đoạn văn bản lừa đảo (cố tình lạm dụng viết hoa toàn bộ và dùng hàng loạt dấu chấm than !!!).	Cảnh báo đỏ (Tin giả). Động cơ Giải thích phải vạch trần được thủ thuật hình thức này.	Thanh đánh giá báo động đỏ phần "Thống kê hình thức".	PASS

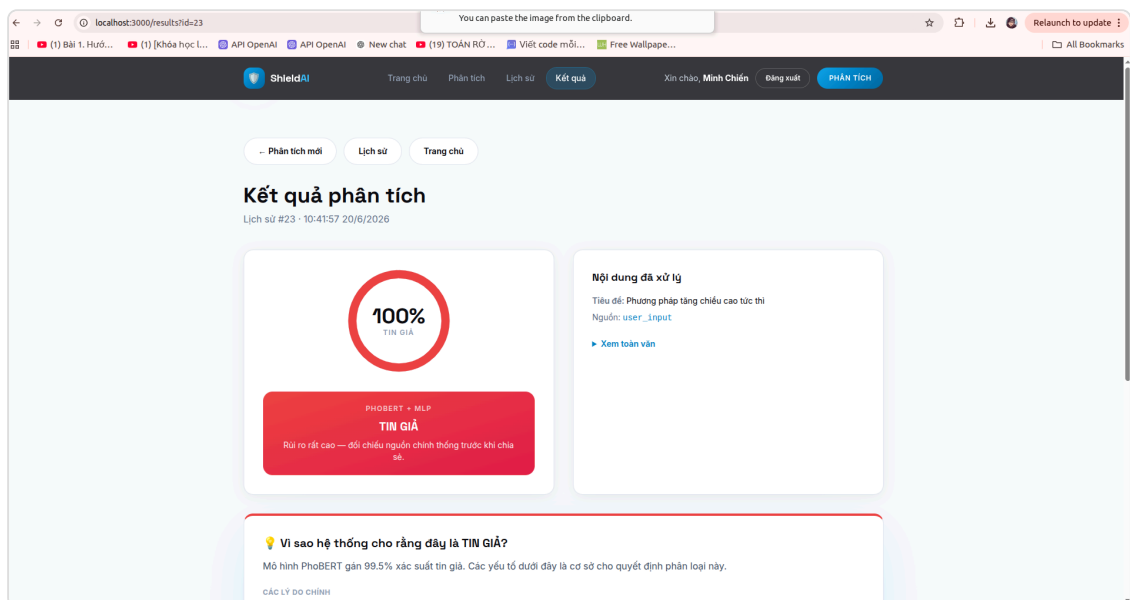
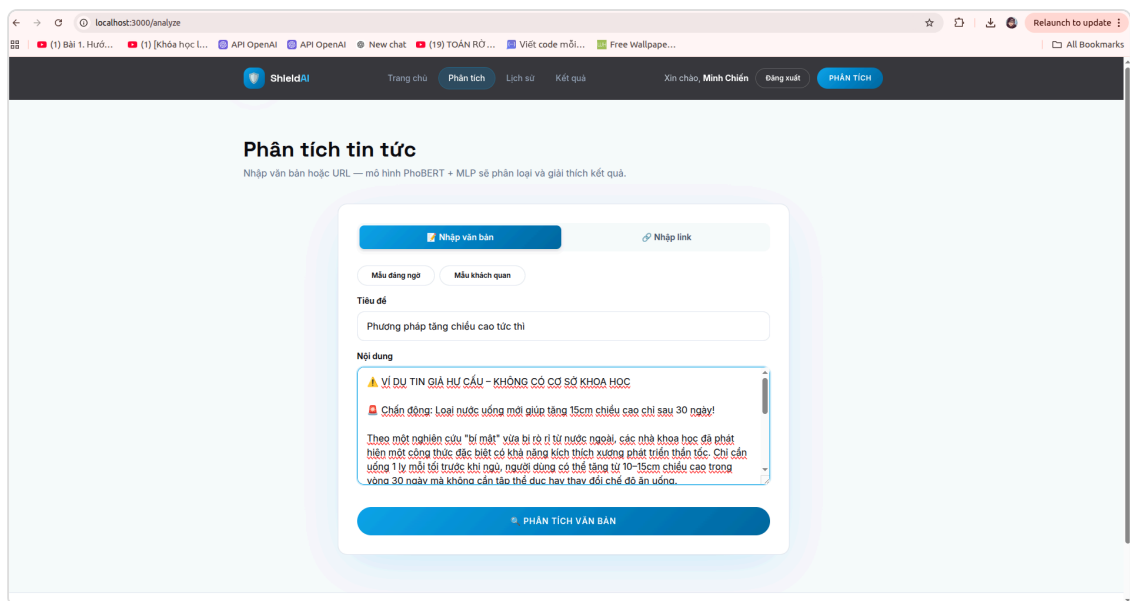
Dưới đây là một số hình ảnh thực tế minh họa cho các kịch bản kiểm thử hộp đen đã được thực thi và xác nhận trên môi trường Frontend:

Hình 4.9. Minh họa Kịch bản TC_01: Phân tích bài báo chính thống (0% Tin giả)





Hình 4.10. Minh họa Kịch bản TC_03: Bóc trần thủ thuật lạm dụng hình thức (100% Tin giả)



4.5. Kết luận chương

Chương 4 đã phản ánh toàn bộ quá trình đưa các bản thiết kế từ trên giấy vào môi trường triển khai thực tế. Các kết quả thực nghiệm đạt được đã chứng minh tính đúng đắn của thuật toán AI. Bên cạnh đó, một số kịch bản kiểm thử (Test cases) cơ bản đã được thiết lập để đảm bảo luồng hoạt động tối thiểu của nền tảng phần mềm hoàn chỉnh, tuy nhiên quá trình kiểm thử phần mềm không phải là trọng tâm khoa học của chương này. Các thành quả này đã đáp ứng đầy đủ yêu cầu nghiệp vụ đặt ra, làm tiền đề để rút ra các kết luận tổng kết và định hướng ở Chương 5.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trải qua quá trình nghiên cứu lý thuyết chuyên sâu và thực nghiệm kỹ thuật nghiêm ngặt, đồ án đã hoàn thành khả quan mục tiêu trọng tâm ban đầu: **Nghiên cứu và xây dựng thành công công cụ phát hiện tin giả tiếng Việt bằng PhoBERT Text-only + MLP tích hợp Động cơ giải thích độc lập (Heuristic Explanation)**. Các kết quả đạt được của đồ án không chỉ dừng lại ở mức độ thử nghiệm mô hình mà đã hoàn thiện thành một sản phẩm phần mềm trọn vẹn (End-to-end System) với các điểm nhấn cốt lõi sau:

- **Về mặt nghiên cứu mô hình:** Đồ án đã loại bỏ thành công sự phụ thuộc phức tạp vào các kiến trúc lai ghép (Hybrid) nặng nề. Việc áp dụng kiến trúc **PhoBERT Text-only + MLP** chuyên biệt đã chứng minh được tính hiệu quả. Khả năng phân tích ngữ nghĩa sâu của PhoBERT kết hợp với mạng phân loại MLP đa tầng (128, 64) đã mang lại các chỉ số thực nghiệm rất đáng kể (F1-Score đạt 93,71%, ROC-AUC đạt 98,50%) trên tập **10.609 mẫu** (hold-out 30% $\approx$ 3.183 mẫu), đạt mức hiệu suất rất cạnh tranh mà không cần đến các kiến trúc mạng quá phức tạp.
- **Về mặt công nghệ và kiến trúc phần mềm:** Hệ thống đã áp dụng kiến trúc phân hiện đại Client-Server. Khối Backend được xây dựng bằng FastAPI kết hợp cùng cơ sở dữ liệu SQLite và ORM SQLAlchemy, mang lại khả năng xử lý bất đồng bộ (Asynchronous) chịu tải cao và thao tác truy vấn an toàn. Khối Frontend sử dụng Next.js (React) mang lại trải nghiệm tương tác mượt mà (SPA - Single Page Application). Toàn bộ hệ thống được kết nối mạch lạc, hỗ trợ linh hoạt hai phương thức: tự động cào dữ liệu (Web Scraping) từ URL bài báo trực tuyến, hoặc phân tích trực tiếp từ đoạn văn bản thô do người dùng cung cấp.
- **Về mặt trải nghiệm người dùng (UX) và tính minh bạch (XAI):** Thay vì chỉ trả về kết quả nhị phân, hệ thống phân chia kết quả thành 3 ngưỡng cảnh báo (Tin thật, Đáng ngờ, Tin giả), đồng thời tự động sinh ra các câu văn tiếng Việt giải thích lý do cảnh báo nhờ Động cơ Heuristics. Điều này giúp người dùng dễ dàng nhận biết các yếu tố lừa đảo, từ đó nâng cao "sức đề kháng" thông tin số của bản thân.
- **Về mặt kỹ nghệ phần mềm:** Quá trình phát triển dự án tuân thủ nghiêm ngặt các quy chuẩn của ngành Công nghệ Phần mềm. Mã nguồn được tổ chức theo mô hình tách biệt mối quan tâm (Separation of Concerns), đi kèm với bộ **42 kịch bản kiểm thử tự động** sử dụng Pytest (Bảng 4.3, 4.4). Việc 100% các kịch bản kiểm thử đều vượt qua (42/42 PASSED) đã minh chứng cho độ tin cậy và tính ổn định của hệ thống.

Nhìn chung, tiểu luận đã cung cấp một giải pháp khả thi và hiệu quả cho bài toán phát hiện tin giả trên không gian mạng Việt Nam, vừa đảm bảo tính hàn lâm khoa học trong thiết kế thuật toán, vừa đáp ứng được các tiêu chuẩn kỹ thuật khắt khe của một ứng dụng phần mềm thực tiễn.

5.2. Đóng góp của tiểu luận

Đồ án không tập trung vào việc đề xuất một kiến trúc trí tuệ nhân tạo mới, mà hướng tới việc giải quyết một bài toán thực tiễn thông qua kỹ nghệ phần mềm và ứng dụng AI. Những đóng góp cốt lõi và giá trị thực tiễn của tiểu luận có thể được đúc kết qua bốn khía cạnh sau đây:

- **Ứng dụng hiệu quả AI cho tiếng Việt:** Đồ án đã vận dụng thành công mô hình ngôn ngữ PhoBERT kết hợp với mạng đa tầng MLP vào bài toán phát hiện tin giả tiếng Việt, đạt được kết quả phân loại khả quan trên tập dữ liệu ngữ liệu đặc thù.
- **Xây dựng hệ thống hoàn chỉnh (End-to-end System):** Không chỉ dừng lại ở việc huấn luyện mô hình học máy trên môi trường thử nghiệm, đồ án đã phát triển thành công một hệ thống phần mềm hoàn chỉnh bao gồm quy trình thu thập dữ liệu tự động, cơ sở dữ liệu lưu trữ, kiến trúc Backend API xử lý bất đồng bộ và giao diện Frontend.
- **Tích hợp Động cơ Giải thích (Explanation Engine):** Việc xây dựng Động cơ giải thích độc lập dựa trên luật (Rule-based Heuristics) chạy song song với mô hình học sâu giúp cung cấp

những lý do cảnh báo trực quan bằng tiếng Việt, từ đó góp phần cải thiện khả năng giải thích và tính minh bạch của mô hình.

- **Triển khai thành sản phẩm ShieldAI:** Đồ án đã đóng gói toàn bộ các thuật toán và module kỹ thuật phức tạp thành một nền tảng Web có tên gọi ShieldAI hỗ trợ người dùng cuối, sẵn sàng hỗ trợ cộng đồng trong việc kiểm chứng thông tin trên không gian mạng.

5.3. Hạn chế của đề tài

Mặc dù đã đạt được những kết quả khả quan cả về mặt lý thuyết lẫn thực tiễn, đồ án vẫn còn tồn tại một số điểm hạn chế mang tính khách quan và chủ quan, cụ thể như sau:

- **Sự phụ thuộc một phần vào kỹ thuật Web Scraping:** Mặc dù hệ thống đã bổ sung tính năng nhập văn bản trực tiếp để khắc phục, tuy nhiên tính năng tự động thu thập dữ liệu (sử dụng BeautifulSoup) vẫn là một thành phần quan trọng nhằm tối ưu UX. Nếu các tòa soạn báo điện tử tiến hành tái cấu trúc (Redesign) giao diện toàn diện, hoặc kích hoạt các lớp bảo mật chống Bot cực đoan như Cloudflare, module này sẽ tạm thời mất đi khả năng tự động lấy nội dung bài viết thông qua URL.
- **Tiêu tốn tài nguyên phần cứng (Hardware Constraints):** Bản chất của mô hình ngôn ngữ tiên huấn luyện PhoBERT (dù là phiên bản Base) vẫn yêu cầu tải vào bộ nhớ hàng triệu tham số. Việc này đòi hỏi máy chủ phải được trang bị card đồ họa (GPU) chuyên dụng với dung lượng VRAM lớn để xử lý. Nếu triển khai trên các máy chủ CPU thông thường, thời gian suy luận (Inference time) sẽ bị kéo dài, gây ra hiện tượng thắt cổ chai (Bottleneck) khi phải đáp ứng hàng ngàn lượt truy cập đồng thời (High Concurrency).
- **Giới hạn về loại hình dữ liệu (Multimedia Limitation):** Dữ liệu phân tích hiện tại của hệ thống tập trung vào văn bản (Text-only). Điều này dẫn đến việc hệ thống chưa có khả năng bóc tách hình ảnh, âm thanh hoặc video giả mạo (Deepfake). Các chiến dịch tin giả hiện đại thường xuyên sử dụng hình ảnh cắt ghép để tăng tính thuyết phục, và đây là một điểm mù mà PhoBERT không thể giải quyết được.
- **Thiếu khả năng phân tích Đa phương thức (Multimodal Analysis):** Trong thời đại số, tin giả thường không chỉ dựa vào văn bản mà còn bị cắt ghép, lồng ghép vào những hình ảnh giả mạo (Photoshopped) hoặc video deepfake. Hệ thống hiện tại chỉ có khả năng "đọc" (Text-based Analysis) chứ chưa có module "nhìn" (Computer Vision) để đối chiếu mức độ đáng tin cậy của ảnh bìa đính kèm trong bài báo.

5.4. Hướng phát triển

Nhằm khắc phục những hạn chế hiện tại và đưa hệ thống ShieldAI trở thành một nền tảng thương mại hóa hoặc ứng dụng rộng rãi trong cộng đồng, đồ án đề xuất các định hướng phát triển cốt lõi trong tương lai như sau:

- **Nâng cấp năng lực Thu thập dữ liệu (Data Ingestion):** Phát triển các module thu thập dữ liệu thông minh hơn thông qua việc tích hợp các API chính thức của các nền tảng mạng xã hội lớn (Facebook Graph API, X/Twitter API, Telegram Bot API). Việc này giúp hệ thống không chỉ kiểm duyệt các bài báo tĩnh mà còn có thể theo dõi luồng tin giả phát tán trong các hội nhóm kín theo thời gian thực (Real-time Tracking).
- **Tích hợp phân tích Đa phương thức (Multimodal AI):** Nâng cấp kiến trúc mạng nơ-ron từ đơn phương thức (Text-only) sang đa phương thức bằng cách tích hợp thêm các mô hình Thị giác Máy tính (Computer Vision) như ResNet hoặc ViT (Vision Transformer). Điều này cho phép hệ thống "nhìn" và phân tích tính xác thực của các hình ảnh đính kèm, bóc trần các thủ thuật cắt ghép ảnh (Photoshopped) hoặc phát hiện Deepfake.
- **Tối ưu hóa Hạ tầng Triển khai (Cloud & Microservices):** Để giải quyết bài toán chi phí cao, toàn bộ mã nguồn Backend và mô hình PhoBERT [1] cần được đóng gói (Containerization) bằng Docker và điều phối bằng Kubernetes. Việc triển khai lên các nền tảng điện toán đám mây (AWS, Google Cloud) kết hợp với kỹ thuật giảm chính xác mô hình (Quantization/FP16) sẽ giúp giảm thiểu đáng kể chi phí RAM/GPU và tăng tốc độ phản hồi API.

- **Học tăng cường và Học liên tục (Active Learning):** Tích hợp cơ chế phản hồi từ người dùng (User Feedback Loop) trực tiếp vào quy trình huấn luyện. Khi hệ thống phân loại sai và được người dùng (hoặc chuyên gia) "cầm cò" báo cáo, dữ liệu này sẽ được thu thập vào kho dữ liệu mới để mô hình tự động cập nhật lại trọng số (Retrain) định kỳ. Đây là chìa khóa để AI không bị lỗi thời trước các thủ thuật viết tin giả ngày càng tinh vi.

TÀI LIỆU THAM KHẢO

- [1] D. Q. Nguyen and A. T. Nguyen, "PhoBERT: Pre-trained language models for Vietnamese," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037-1042.
- [2] S. Lee, "Vietnamese Medical Fake News Dataset," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/leviettrieu369/vietnamese-medical-fake-news-dataset>. [Accessed: Jun. 12, 2026].
- [3] X. Zhou and R. Zafarani, "A Survey of Fake News: Fundamental Theories, Detection Methods, and Opportunities," *ACM Computing Surveys (CSUR)*, vol. 53, no. 5, pp. 1-40, 2020.
- [4] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, "Fake News Detection on Social Media: A Data Mining Perspective," *SIGKDD Explor. Newsl.*, vol. 19, no. 1, pp. 22-36, 2017.
- [5] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, 2017, pp. 4765-4774.
- [6] A. Vaswani et al., "Attention is All you Need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998-6008.
- [7] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38-45.
- [8] S. Ramírez-Gallego, J. R. Romero, and C. García, "Heuristic Explanation in Fake News Detection: A Review," *IEEE Access*, vol. 9, pp. 45210-45225, 2021.
- [9] S. Ramírez-Gallego et al., "Fast Web Scraping and Data Extraction Architecture for NLP Applications," *Journal of Big Data*, vol. 6, no. 1, p. 55, 2019.
- [10] D. U. Nguyen, K. V. Nguyen, and N. L.-T. Nguyen, "Vietnamese Text Classification using Deep Learning Architectures," in *Proceedings of the International Conference on Asian Language Processing (IALP)*, 2021, pp. 102-107.
- [11] V. T. N. Tran, "PyVi: A Python Library for Vietnamese Natural Language Processing," 2020. [Online]. Available: <https://pypi.org/project/pyvi/>. [Accessed: Jun. 12, 2026].
- [12] S. Ramírez, "FastAPI framework, high performance, easy to learn, fast to code, ready for production," 2024. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Jun. 12, 2026].
- [13] Vercel, "Next.js by Vercel - The React Framework for the Web," 2024. [Online]. Available: <https://nextjs.org/>. [Accessed: Jun. 12, 2026].
- [14] Pytest-dev Team, "pytest: helps you write better programs," 2024. [Online]. Available: <https://docs.pytest.org/>. [Accessed: Jun. 14, 2026].

PHỤ LỤC

Phụ lục A: Mã nguồn chương trình

Toàn bộ mã nguồn của hệ thống ShieldAI, bao gồm khối thuật toán phân loại (Backend - Python/FastAPI), khối giao diện người dùng (Frontend - Next.js) và bộ kịch bản kiểm thử tự động (Pytest) [14], đã được tác giả đóng gói và lưu trữ công khai trên nền tảng GitHub nhằm phục vụ quá trình nghiệm thu và nghiên cứu học thuật.

Địa chỉ truy cập mã nguồn (Repository): <https://github.com/Usunase/DoAnTotNghiep.git>

Phụ lục B: Hướng dẫn sử dụng

Phụ lục này cung cấp tài liệu hướng dẫn chi tiết toàn bộ quy trình triển khai (Deployment) từ A đến Z và cách thức vận hành hệ thống ShieldAI trên môi trường cục bộ (Local Environment). Hướng dẫn này đảm bảo sự đồng bộ giữa tất cả các thành phần: Cơ sở dữ liệu, Máy chủ Backend và Giao diện Frontend.

B.1. Yêu cầu Hệ thống (System Requirements)

Để đảm bảo hệ thống vận hành trơn tru, máy tính cá nhân hoặc máy chủ cần đáp ứng các thông số kỹ thuật tối thiểu sau:

- **Hệ điều hành:** Windows 10/11, macOS hoặc Linux (Ubuntu 20.04+).
- **Nền tảng phần mềm:** Đã cài đặt sẵn **Python 3.10** (trở lên) và **Node.js 18.x** (trở lên).
- **Phần cứng (Khuyến nghị):** CPU đa nhân (Core i5/Ryzen 5), RAM tối thiểu 8GB (ưu tiên 16GB để chạy mô hình AI mượt mà) và ổ cứng SSD. *Không bắt buộc phải có GPU (Card rời) ở môi trường suy luận (Inference), hệ thống tự động tính toán dự phòng (fallback) về CPU.*

B.2. Hướng dẫn Khởi chạy Toàn bộ Hệ thống

Hệ thống ShieldAI tuân thủ kiến trúc phân tách (Decoupled Architecture). Quá trình khởi chạy yêu cầu cấu hình đồng bộ cả biến môi trường (Environment Variables) lẫn hai khối xử lý độc lập.

Bước 1: Thiết lập Biến môi trường và Cấu hình (Configuration)

Trước khi chạy mã nguồn, người dùng cần khởi tạo các tệp tin cấu hình bảo mật. Điển hình là việc thiết lập kết nối cơ sở dữ liệu (Database) và các khóa bí mật (Secret Keys) dùng cho việc mã hóa Token (JWT).

```
# Di chuyển vào thư mục Backend
cd backend

# Nhân bản file cấu hình mẫu thành file môi trường thực tế
cp .env.example .env

# (Tùy chọn) Mở file .env và chỉnh sửa SECRET_KEY hoặc cấu hình Database nếu cần
```

Bước 2: Khởi động khối Máy chủ API (Backend)

Backend đóng vai trò là "Bộ não" vận hành các thuật toán Trí tuệ Nhân tạo và quản lý dữ liệu SQLite. Mở công cụ Terminal và thực hiện lần lượt các lệnh sau:

```
# 1. Chắc chắn bạn đang ở thư mục Backend
cd backend

# 2. Tạo môi trường ảo hóa (Virtual Environment) để cô lập thư viện
python -m venv venv

# 3. Kích hoạt môi trường ảo (Windows: venv\Scripts\activate)
source venv/bin/activate

# 4. Cài đặt các thư viện lõi (FastAPI, PyTorch, Transformers, v.v.)
pip install -r requirements.txt

# 5. Kích hoạt Máy chủ bằng Uvicorn
uvicorn main:app --reload --port 8000
```

Khi Terminal xuất hiện thông báo "*Application startup complete*", máy chủ API đã sẵn sàng nhận kết nối tại địa chỉ nội bộ: **http://localhost:8000**. Giao diện tài liệu API (Swagger UI) tự động sinh ra và có thể truy cập để kiểm thử trực tiếp tại **http://localhost:8000/docs**.

Bước 3: Khởi động khối Giao diện (Frontend)

Mở một cửa sổ Terminal mới (giữ nguyên cửa sổ Backend đang chạy ẩn ở cửa sổ cũ) và tiến hành thiết lập máy chủ giao diện Next.js:

```
# 1. Di chuyển vào thư mục Frontend
cd frontend

# 2. Cài đặt toàn bộ gói phụ thuộc (Dependencies)
npm install

# 3. Chạy máy chủ giao diện ở chế độ Development
npm run dev
```

Giao diện người dùng sẽ lập tức được biên dịch và chạy tại địa chỉ: **http://localhost:3000**. Lúc này, cả hai hệ thống đã được đồng bộ hóa. Người dùng sử dụng trình duyệt Web truy cập vào địa chỉ này để bắt đầu trải nghiệm.

B.3. Hướng dẫn Thao tác Sử dụng (User Manual)

- **Đăng ký và Đăng nhập:** Ngay tại màn hình Cổng thông tin (Home), người dùng mới bấm chọn nút "*Đăng ký*" để tạo tài khoản cá nhân. Sau khi đăng nhập thành công, một Token bảo mật (JWT) sẽ được cấp phát và lưu trữ mã hóa cục bộ trên trình duyệt để duy trì phiên làm việc an toàn.
- **Kiểm chứng Tin tức:** Tại bảng điều khiển trung tâm, người dùng có thể linh hoạt lựa chọn giữa 2 tính năng phân tích (thông qua hệ thống Tab điều hướng):
 - *Tab Nhập văn bản:* Trực tiếp dán toàn bộ đoạn văn bản thô (tiêu đề, nội dung bài đăng Facebook/Zalo) vào khung nhập liệu. Chế độ này hữu ích khi nguồn tin xuất phát từ các mạng xã hội, tin nhắn cá nhân, hoặc khi hệ thống không thể tự động cào dữ liệu qua link.
 - *Tab Nhập Link (URL):* Sao chép đường dẫn bài báo nghi ngờ và dán vào thanh tìm kiếm. Hệ thống sẽ tự động quét, bóc tách cấu trúc HTML để trích xuất nội dung cốt lõi và loại bỏ quảng cáo dư thừa.

Sau khi hoàn tất nhập liệu, nhấn nút "*Phân tích*" để đẩy nội dung vào Động cơ AI.

- **Phân tích Báo cáo Minh bạch (Giải thích Heuristic):** Trong chưa đầy 3 giây, hệ thống trả về kết quả dưới dạng vòng tròn định lượng (Confidence Score). Quan trọng nhất, người dùng cần lướt xuống phần "*Báo cáo Thống kê Hình thức*". Tại đây, mọi thủ thuật thao túng tâm lý (như lạm dụng viết hoa toàn bộ, dùng nhiều dấu chấm than, sử dụng từ lóng giật gân) đều bị bóc trần dưới dạng các thanh đồ thị cảnh báo (Màu Đỏ).
- **Tra cứu và Quản lý Lịch sử:** Người dùng truy cập mục "*Lịch sử của tôi*" trên thanh điều hướng để xem lại kho lưu trữ toàn bộ các bài viết đã phân tích trong quá khứ. Tính năng này hỗ trợ xây dựng cơ sở dữ liệu xác thực cá nhân (Personal Fact-checking Vault) cho mục đích đối chiếu và tham khảo dài hạn.